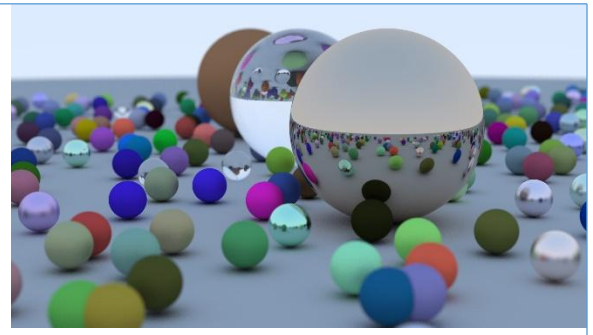
Comp4422



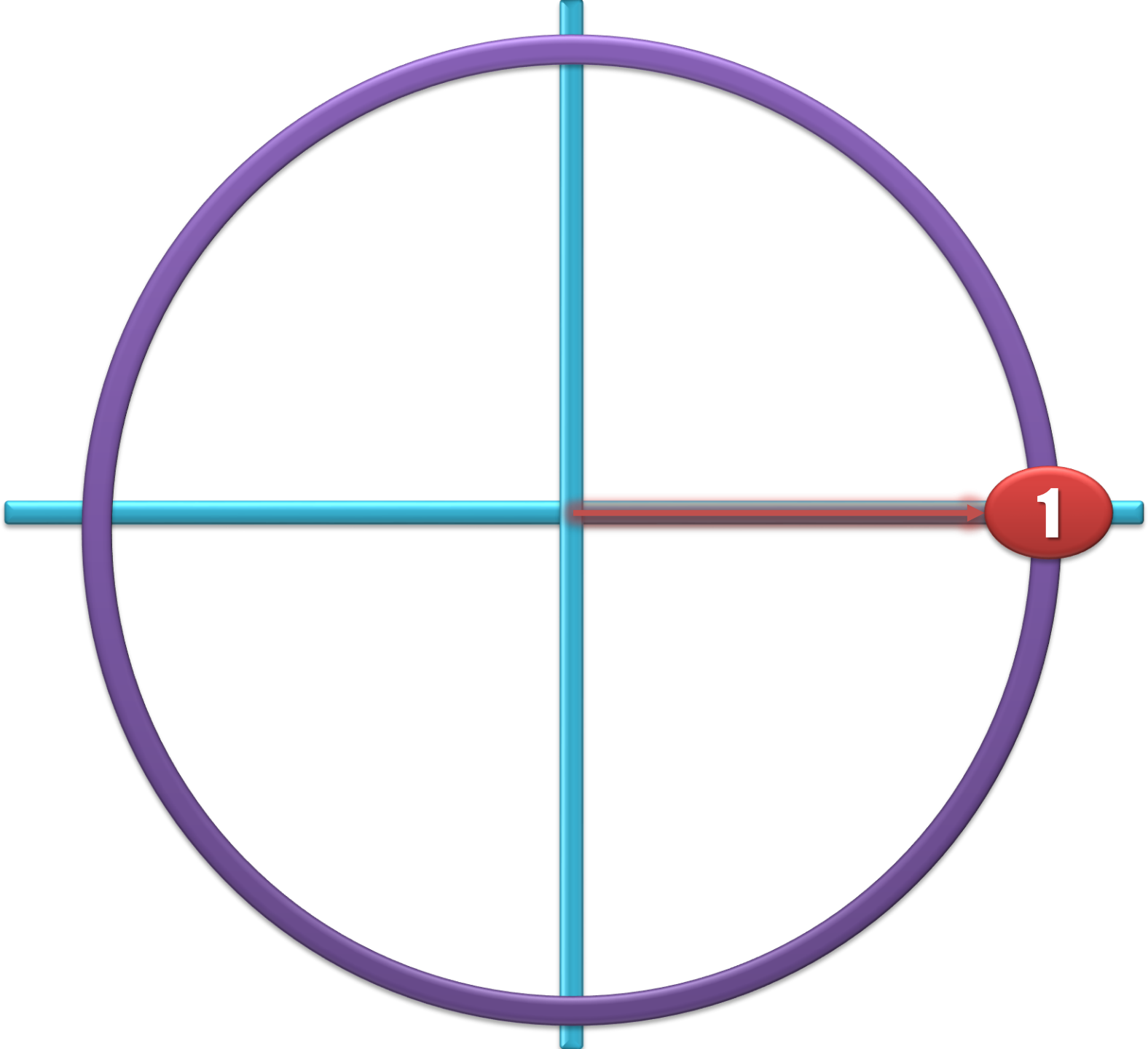
Computer Graphics

Lecture 02:
GPUs



Recap & Roadmap

- W1: images begin with light; the eye is a 3-channel (L/M/S) sensor
- Today: (1) how images are STORED — framebuffers; (2) how they're SHOWN — displays; (3) how they're COMPUTED — GPUs
- **Friday's cliffhanger gets answered: why are monitor pixels exactly R, G, B?**
- Lab link: Lab 1 this week = procedural pixels; you already own a framebuffer in JavaScript


1

Data Flow

CPU 1

- Application
- Modeling Transformations

GPU 2

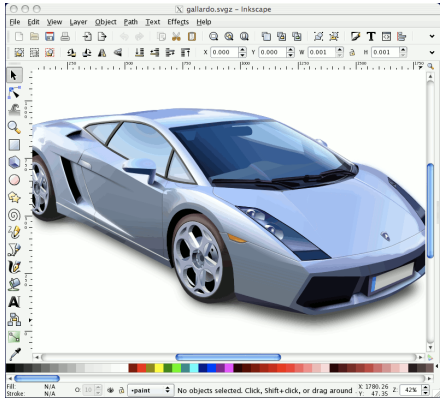
- Viewing, Projection, Clipping
- Vertices and Fragments

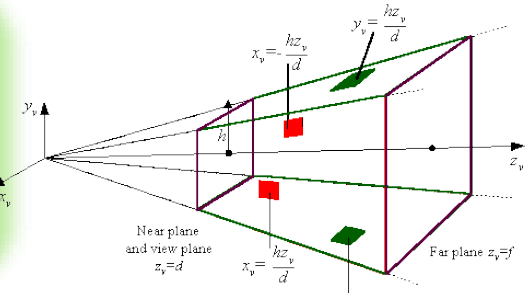
3

- GPU VRAM
- Framebuffer

Display

- Displays

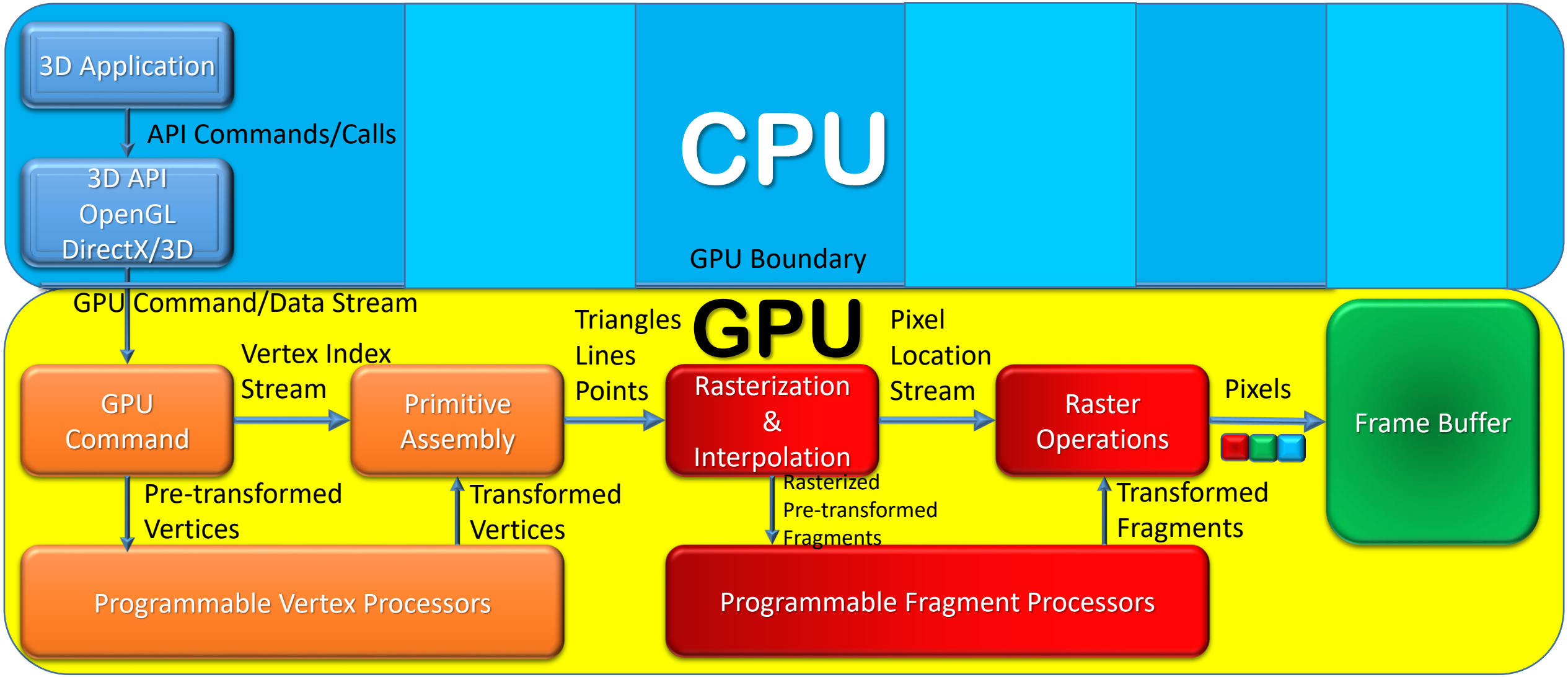








CPU-GPU Boundary



The CPU–GPU Boundary

- Your 3D app never touches the GPU directly — it talks through an API
- APIs: OpenGL · DirectX/3D · Vulkan · Metal · WebGL2 (this course)
- API calls become a command + data stream crossing the bus (PCIe, or shared memory on UMA parts)

Inside the GPU Side of the Boundary

- Vertex index stream → programmable VERTEX processors → transformed vertices
- Primitive assembly → triangles/lines/points
- RASTERIZATION & interpolation → pre-transformed fragments
- Programmable FRAGMENT processors → shaded fragments
- Raster operations (depth/stencil/blend) → framebuffer → pixel location stream

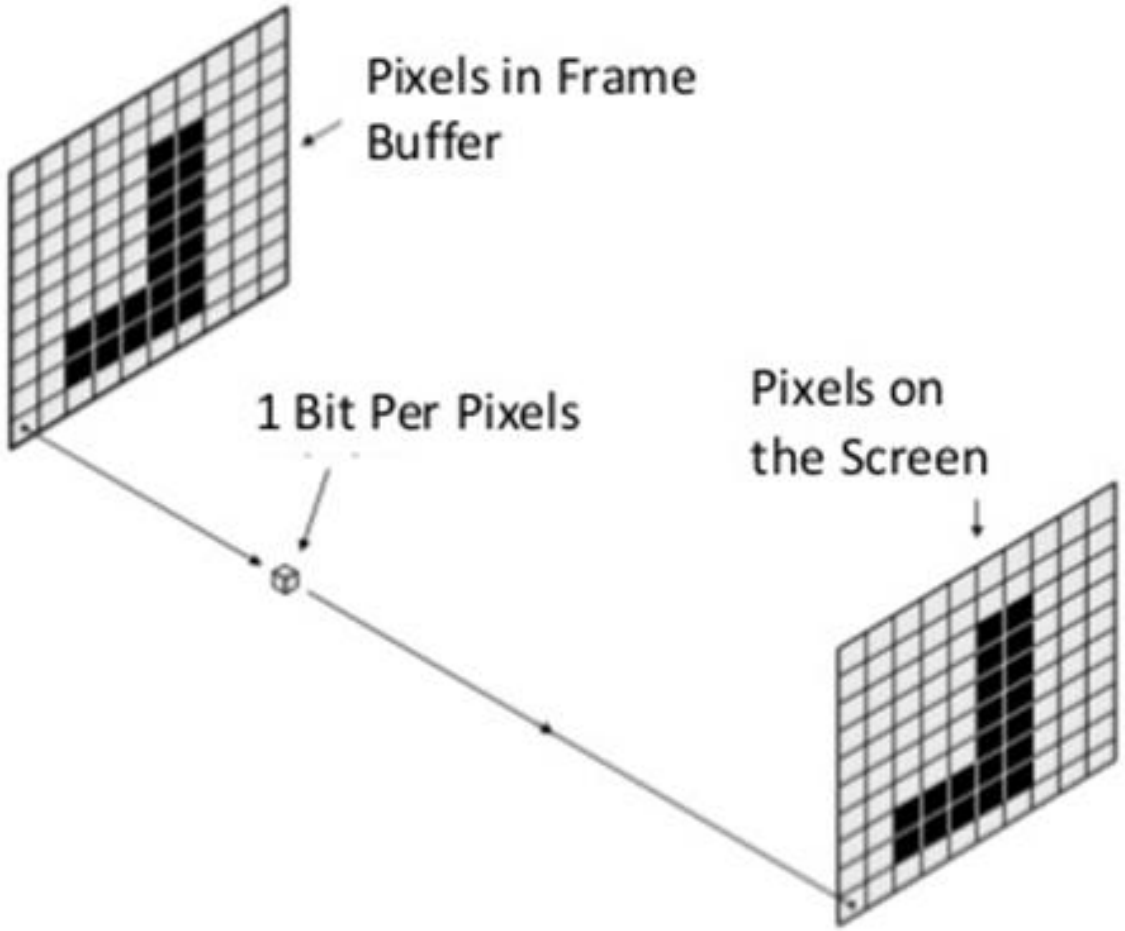
The Framebuffer: Definition

- • A contiguous block of memory holding one colour per pixel
- • Width × Height × Bytes-per-pixel — e.g. 1920×1080×4 B ≈ 8.3 MB per frame
- • Lives in VRAM (discrete GPU) or shared memory (integrated/UMA)
- **You allocated one by hand in Lab 0: ImageData(512, 512) = 1 MB**

Framebuffer Depth: How Many Bits?

- • 1-bit: on/off — the original 'bitmap'
- • 8-bit indexed: palette lookup (GIF still works this way)
- • 24-bit 'true colour': 8+8+8 = 16.7M colours
- • 32-bit RGBA: adds alpha for compositing
- • HDR era: 10-bit per channel (30-bit) panels and fp16 render targets
- **Q: why did 8 bits per channel become the standard?**

1-Bit FrameBuffer

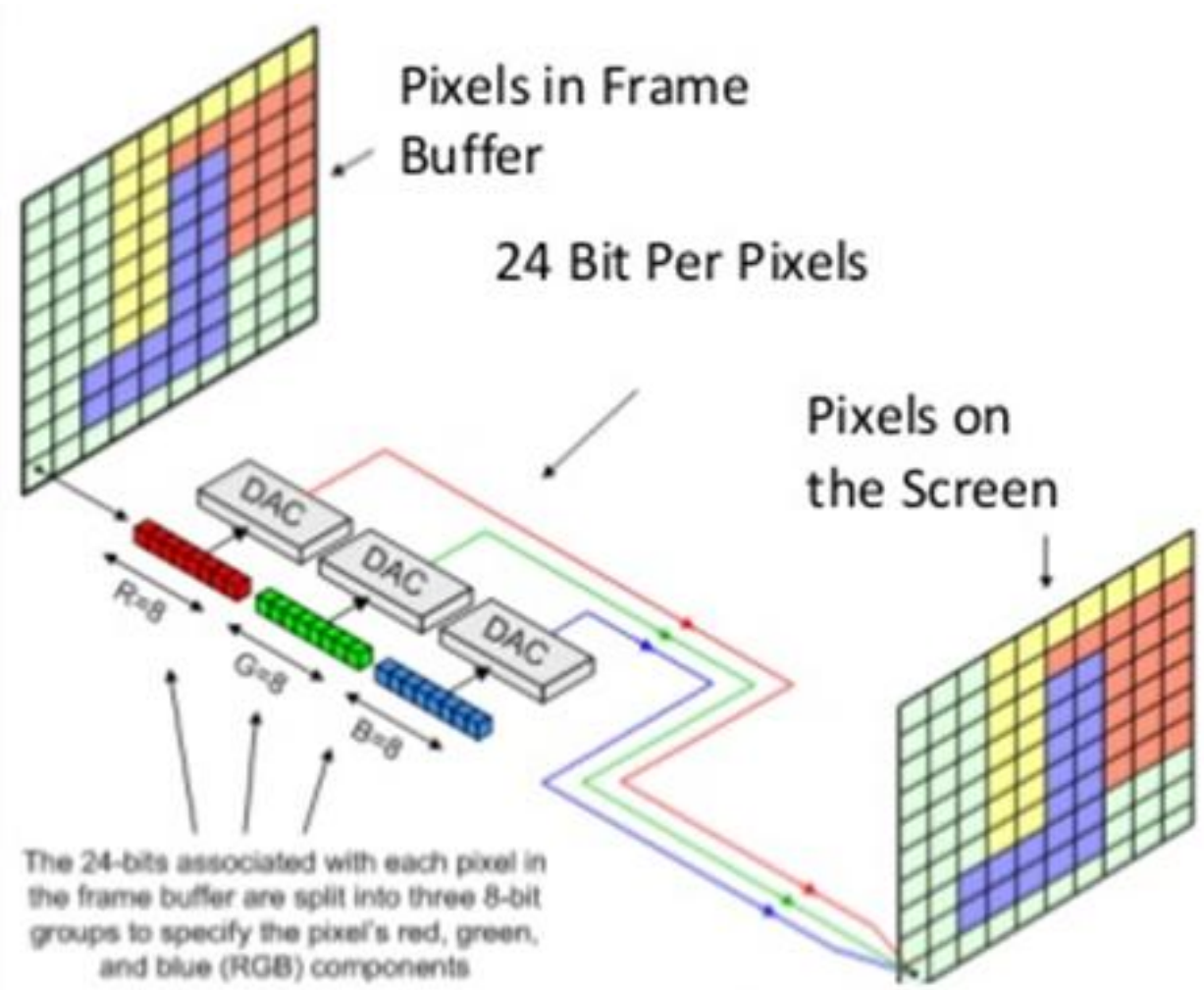


Graphical representation of 1 bit color

More Than Colour: The Buffer Family

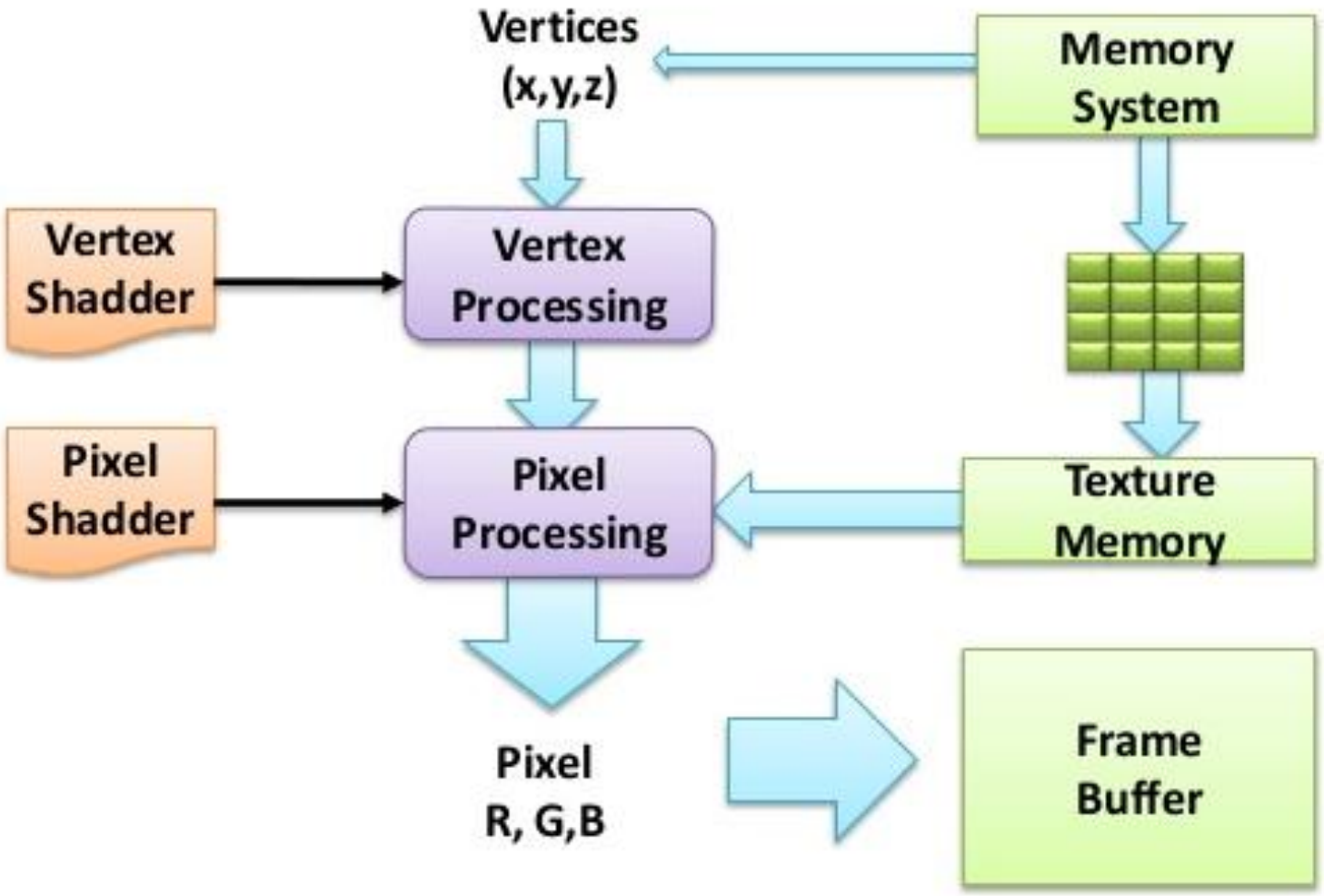
- Colour buffer(s): the image you see
- Depth buffer (z-buffer): one depth value per pixel — W5's star algorithm
- Stencil buffer: per-pixel masks (portals, outlines, shadows)
- **'Framebuffer' in modern APIs = a COLLECTION of attachments, not just colour**

24-Bit FrameBuffer

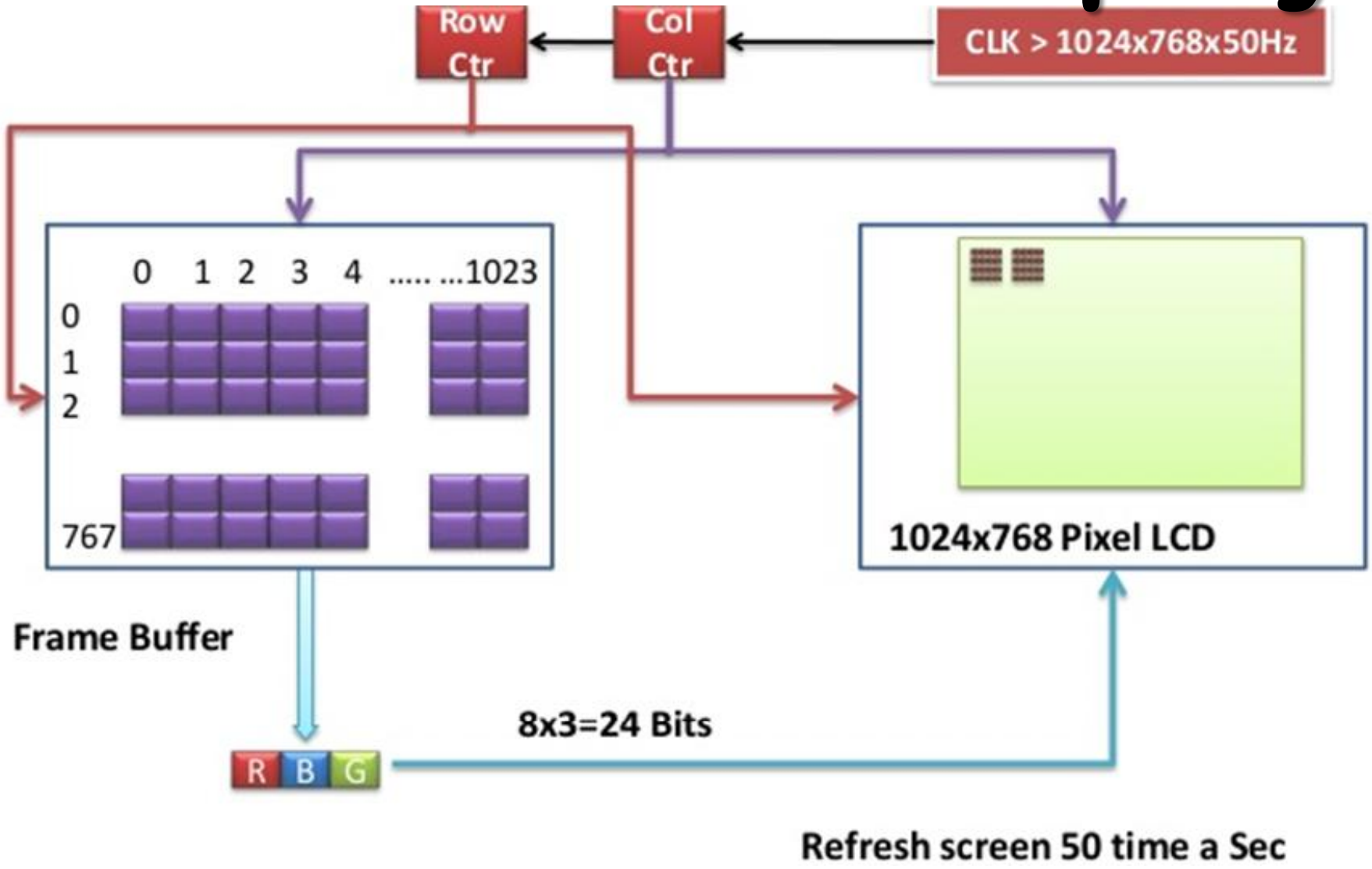


Graphical representation of 24 bit color

Graphics Sub-System

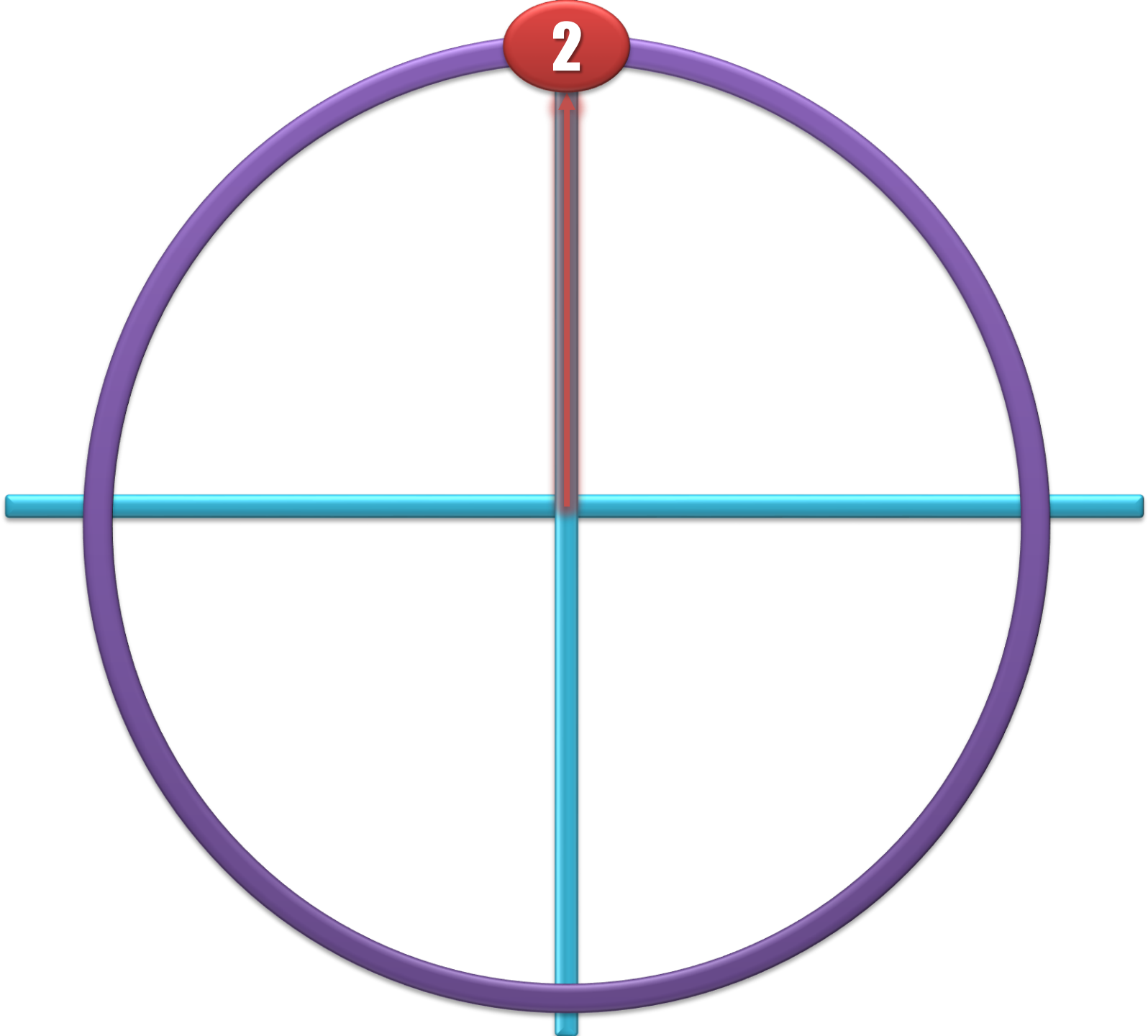


Framebuffer-Display

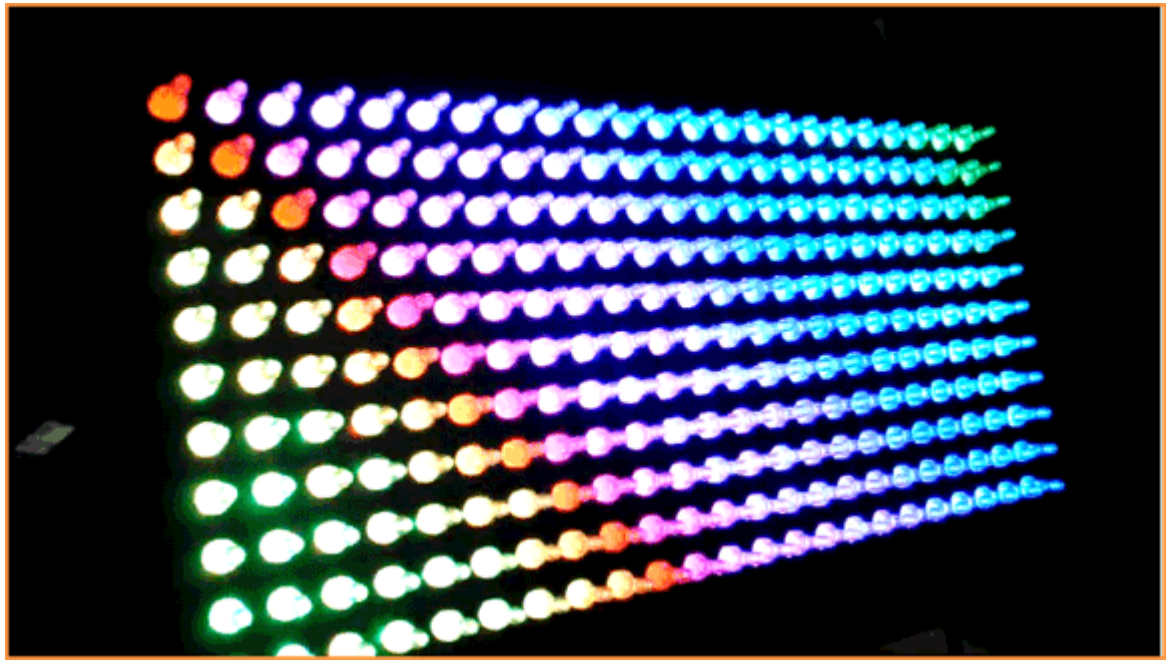


APIs: What They Actually Buy You

- Portability: same code runs on NVIDIA/AMD/Intel/Apple GPUs
- State machine model: set state (buffers, shaders, flags) → then draw
- WebGL2 = OpenGL ES 3.0 brought to the browser — the course's API
- **Mental model from Lab 2: compile shaders, fill buffers, set state, drawArrays**


2

Displays



From framebuffer to photons

Large Displays

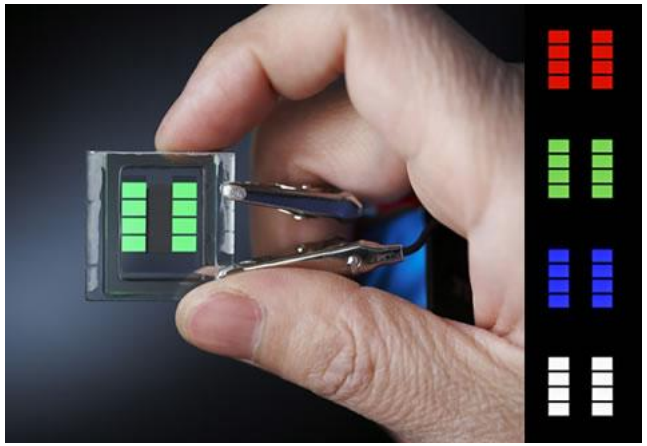




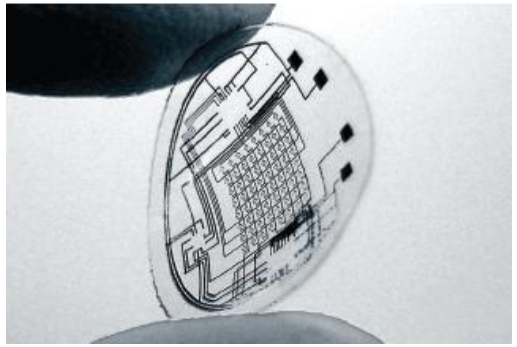
Small Displays

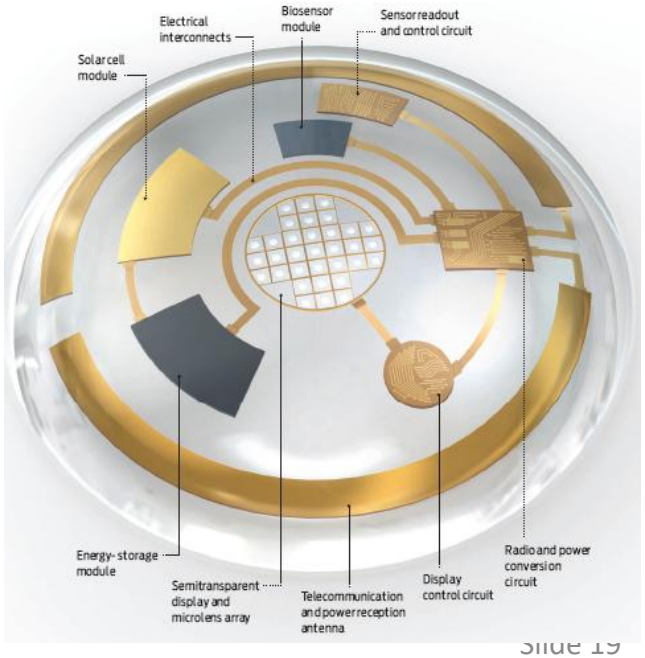
- Small





- And very small

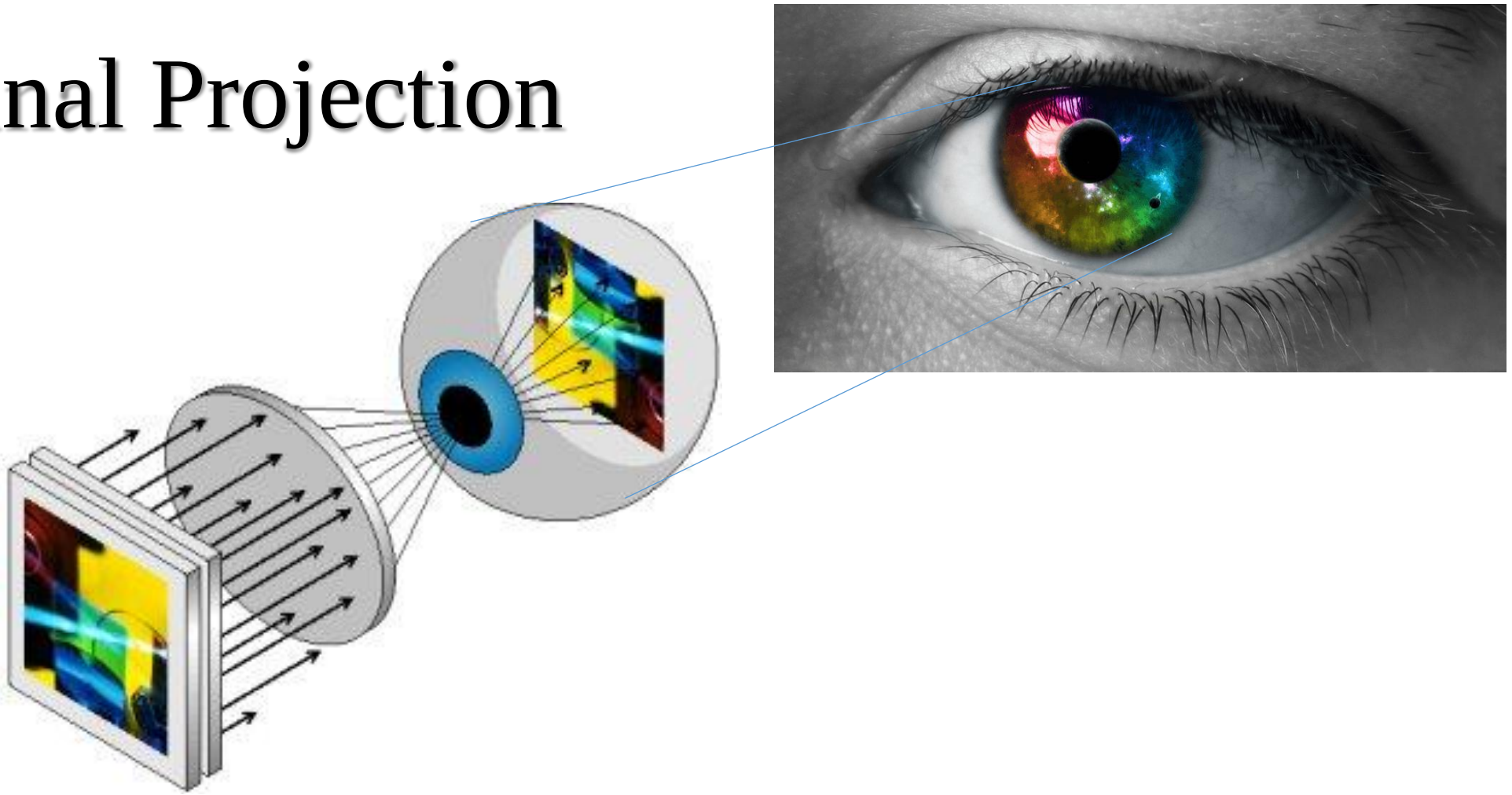




IEEE Spectrum, 2009
Babak A. Parviz
Univ. Washington, Seattle

The Ultimate Display

- Retinal Projection



LCD Displays



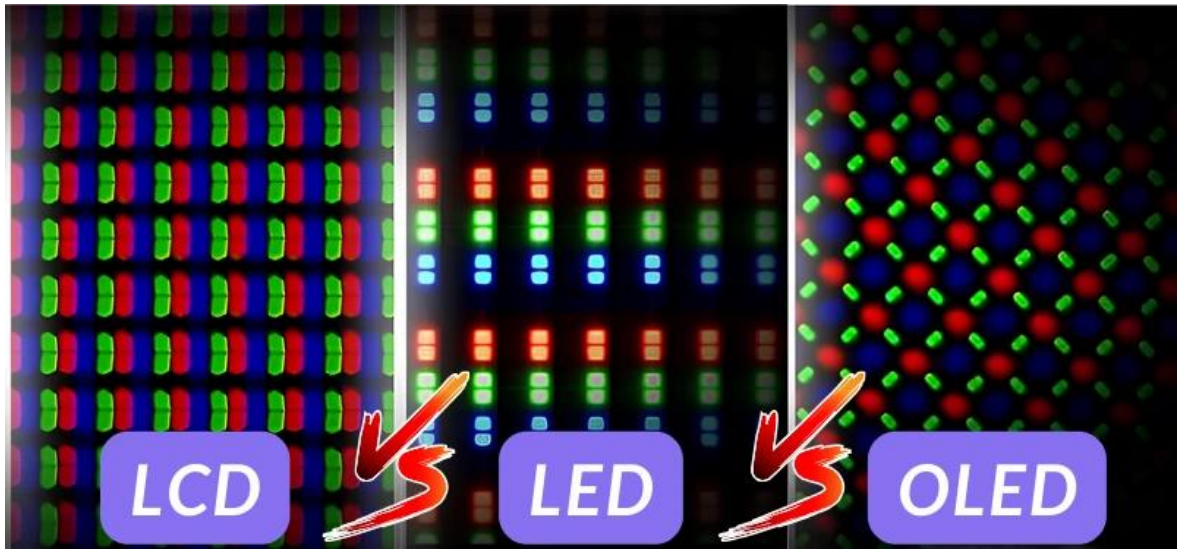
- Most common
- High production volume
- Lower costs
- Energy efficient
- Space efficient

All Flat-Panel Displays Are Raster

- Active matrix: every pixel individually addressable by (x, y)
- Pixel = picture element; sub-pixels = R, G, B
- **Full-circle moment: R, G, B sub-pixels exist because YOUR retina samples light with L, M, S cones**
- The display and the eye are matched 3-channel systems

A Pixel Under the Microscope (Then and Now)

- Sub-pixel geometries vary: RGB stripe (LCD), PenTile (OLED), triangular layouts



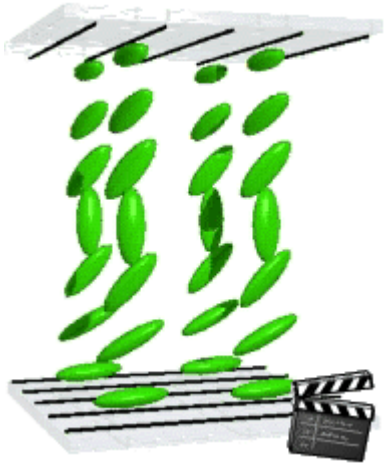
- Sub-pixel rendering (e.g. text anti-aliasing) exploits this geometry

LC = Liquid Crystals

- What are LC's?
  - Substance exhibiting states between a liquid and a solid.
  - Crystals re-align when an electric field is created by applying a voltage between two electrode plates.



Two different States of LC's



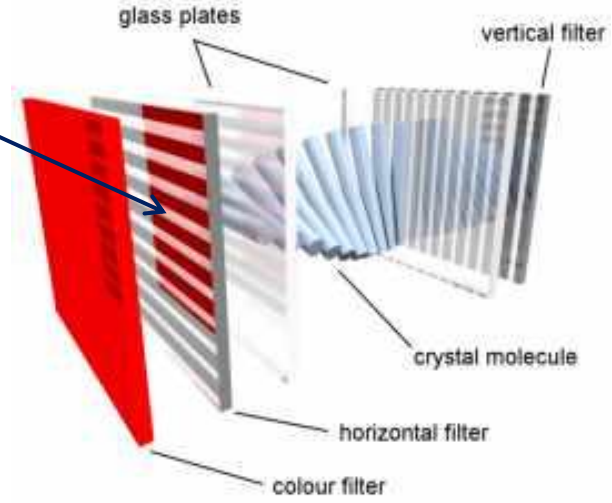
LCD Physics in Three Steps

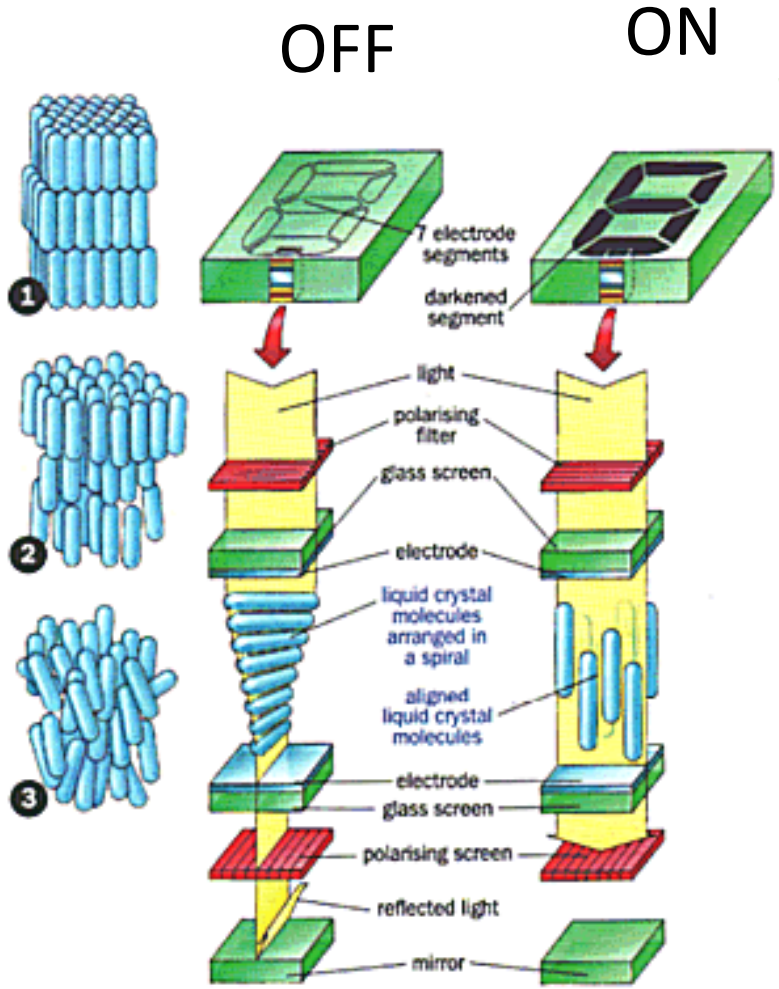
1. Liquid crystals: molecules between liquid and solid states; re-align under applied voltage
2. Sandwiched between crossed polarizers: aligned LC rotates light's polarization → passes; unaligned → blocked
3. A backlight shines through; per-sub-pixel voltage = per-channel brightness

- General design:
  - backlight → polarizer → LC cell → colour filter → polarizer

Basic Principle

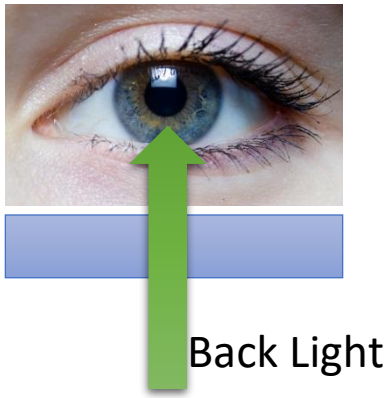
- Nematic Liquid Crystals
- Natural state is to align with grooves





Enhancements

- Transmissive = light from a **backlight** passes once thru the pixels out to your eye. Notebook computers use transmissive displays.



- Reflective = light from the **environment** is reflected at the back of the display and passes back thru the pixels out to your eye.



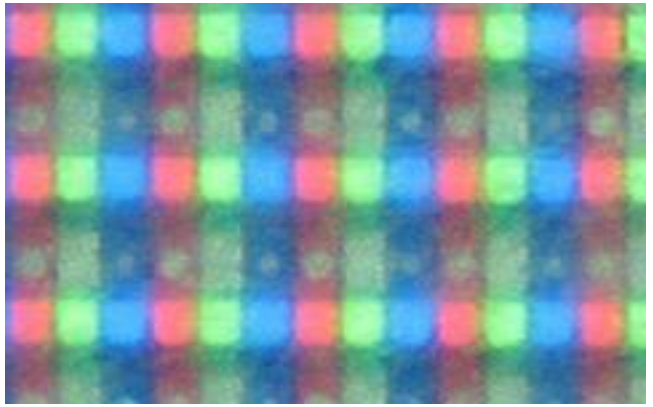
Transflective = transmissive + reflective.

Transmissive, Reflective, Transflective

- Transmissive: backlight passes once through pixels (laptops)
- Reflective: ambient light bounces back through (e-readers, sunlight-readable)
- Transflective: both paths — outdoor-visible phones of the 2000s; survives in niche/industrial gear

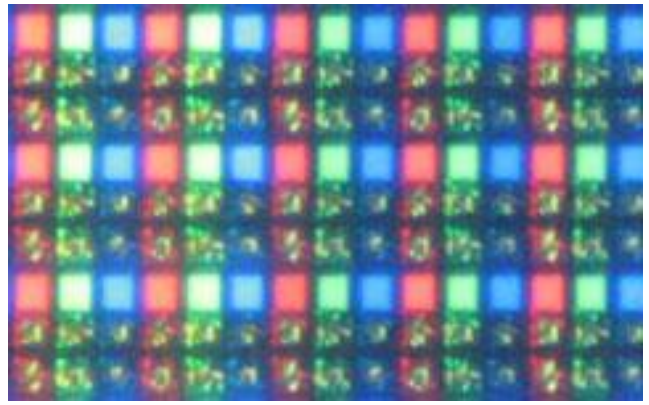
How it looks…

- Different screens under the microscope.



iPod

2.5” 320 x 240 transflective TFT

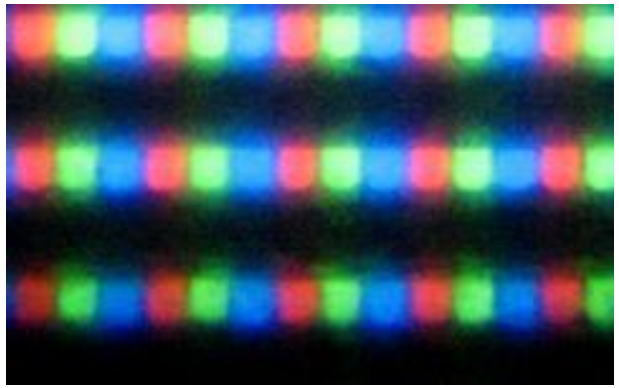


Nokia 6230i

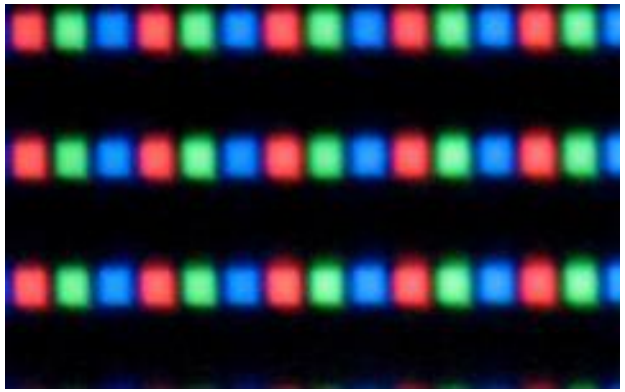
1.7” 208 x 208 transflective TFT

Both TFT LCD displays captured
with a Canon PowerShot SD200 thru a 30X microscope.

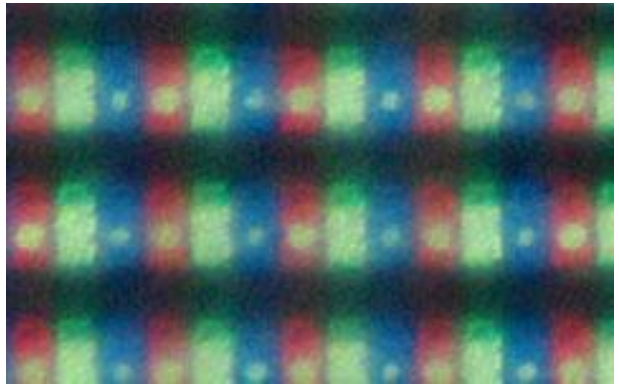
How it looks…



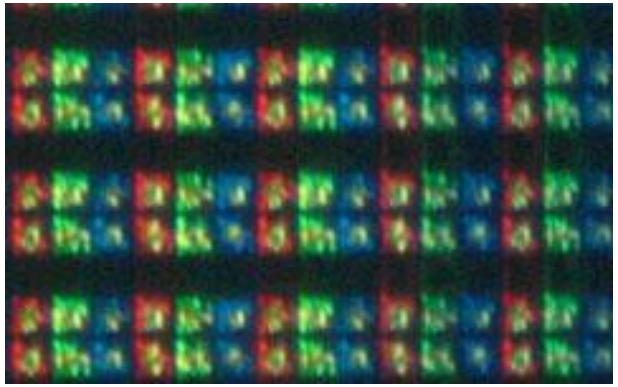
iPod transmissive rows only



Nokia 6230i transmissive only



iPod reflective rows only



Nokia 6230i reflective only

Transflective TFT Display

- Employ two complimentary display techniques to improve visibility across a wider range of ambient light conditions.
- Each pixel has separate transmissive and reflective areas can be seen in the close-up views of a single pixel.

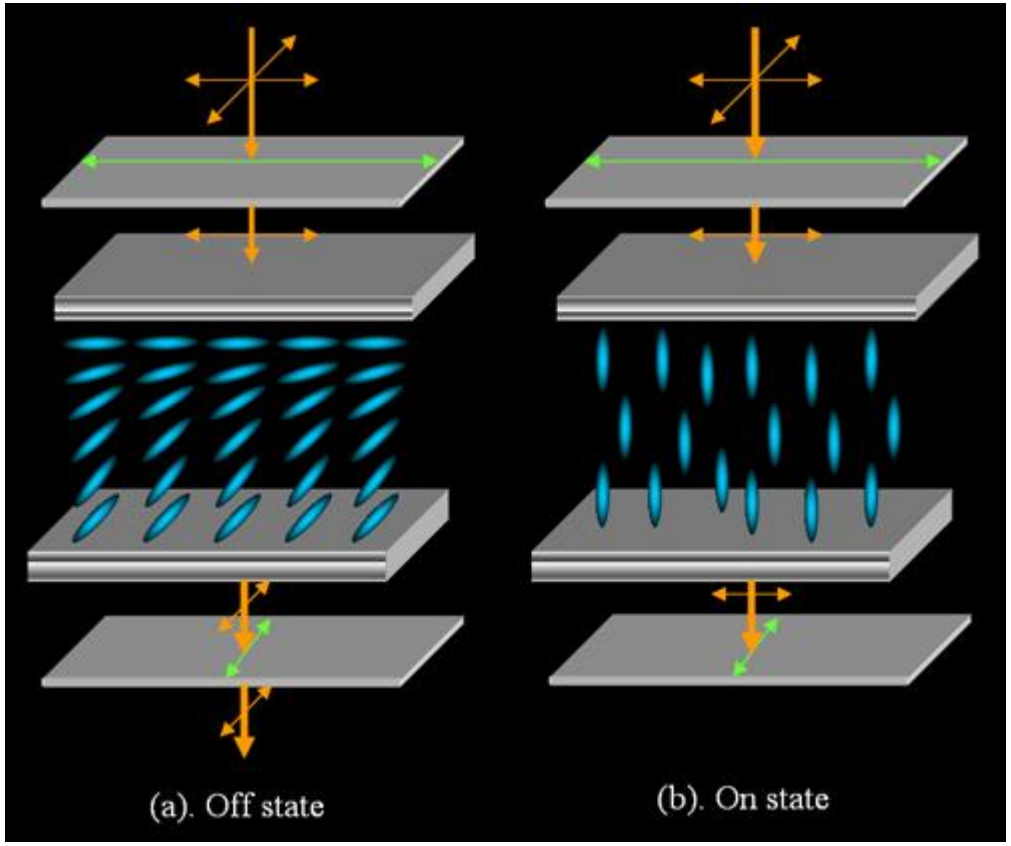
LCD Types

- TN (twisted nematic, 1971): fast, cheap, poor viewing angles
- IPS (in-plane switching, 1996-): voltage in the substrate plane → better colour & angles
- VA (vertical alignment): best static contrast

You don't need the genealogy — you need 'all LCDs = voltage-modulated backlight'

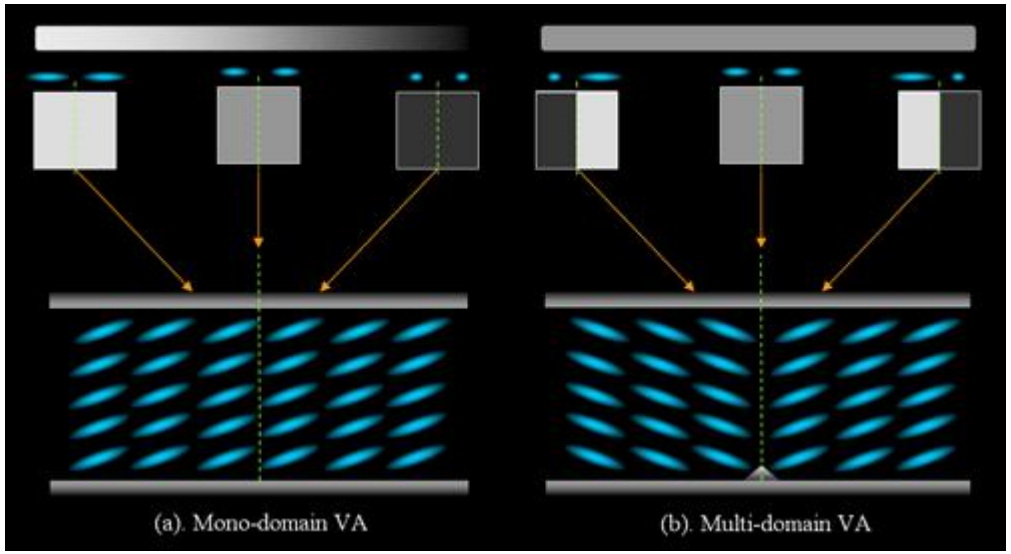
Twisted Nematic

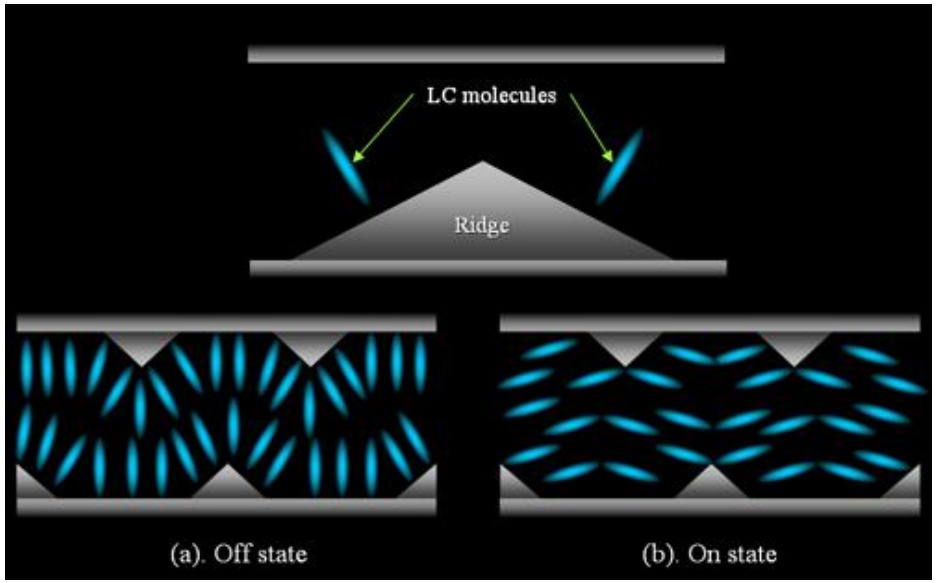
- TN first introduced Schadt, Helfrich and Fergason in 1971
- Traditional LCD



Multi-domain Vertical Alignment

1999 Fujitsu.

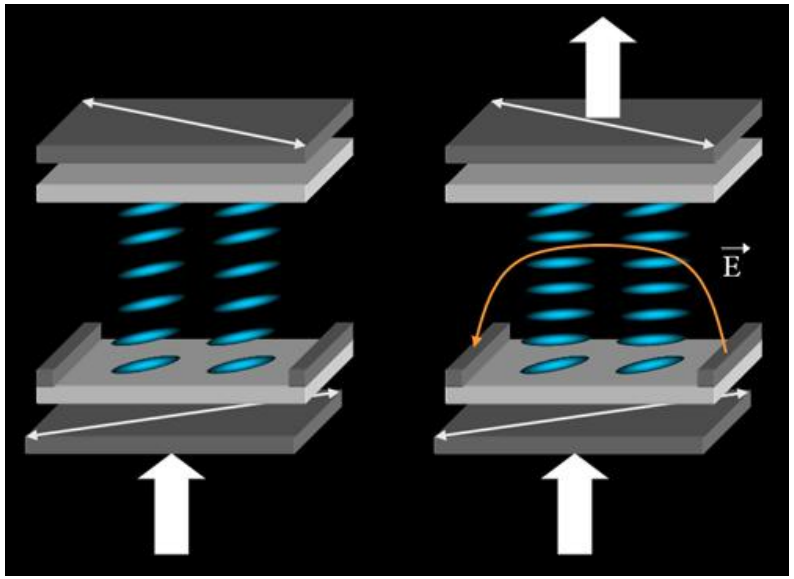




Can give very good viewing angles
Simpler manufacturing – no rubbing

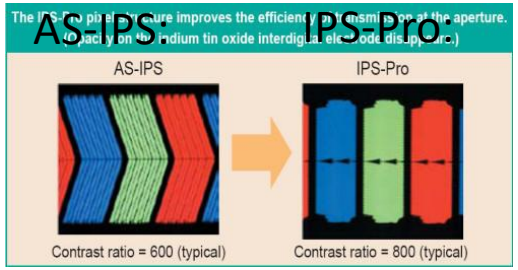
In-Plane Switching

- Voltage is applied in the same plane as the substrate
- Hitachi 1992 and 2002.



- High Image Quality
- Very good contrast
- High brightness
- Wide aperture
- Variations:

AS-IPS: IPS-Pro:



OLED: Emissive Pixels

- No backlight — each sub-pixel is an organic LED emitting its own light
- True black = pixel OFF → 'infinite' contrast; very fast response (~0.1 ms class)
- QD-OLED: blue OLED + quantum-dot colour conversion for purer R/G
- Trade-offs: burn-in risk, brightness limits vs Mini-LED

Mini-LED & Local Dimming

- Still an LCD — but the backlight is thousands of tiny LEDs in zones
- Bright zones bright, dark zones dimmed → HDR without OLED's burn-in

Same lesson as rendering: better locality of light = better image

HDR & Wide Colour Gamut

- SDR: 8-bit, ~100 nits, sRGB gamut
- HDR10/PQ: 10-bit, 1000+ nits peak, Rec.2020 container
- Tone mapping: fitting scene-referred light into display-referred limits

Colour gamuts get their full treatment in W9 (colour models); today: hardware exists beyond sRGB

Refresh Rate & VRR

- Refresh: panel re-scans 60/120/144/240 times per second, frame ready or not
- Mismatch: GPU delivers 73 fps to a 60 Hz panel → tearing or stutter
- VRR (G-Sync / FreeSync / HDMI VRR): panel waits for the GPU — scan-out follows frame completion

Connects to double buffering: VRR moves the swap off the fixed vsync grid

Pixel Density & 'Retina'

- ppi = pixels per inch; beyond ~300 ppi at phone distance the eye stops resolving pixels
- HiDPI consequence: one CSS pixel ≠ one physical pixel — WebGL canvases must scale for devicePixelRatio

Practical lab note: canvas.width vs canvas.clientWidth — your Lab 3+ code must handle this

Near-Eye & Spatial Displays

- VR/AR: micro-OLED panels, ~4K per eye, 90-120 Hz, low persistence
- Why low persistence? Full-persistence panels smear under head motion → sickness
- The 2009 'ultimate display' (direct retinal projection) survives as AR research — shipping XR gets closer each generation



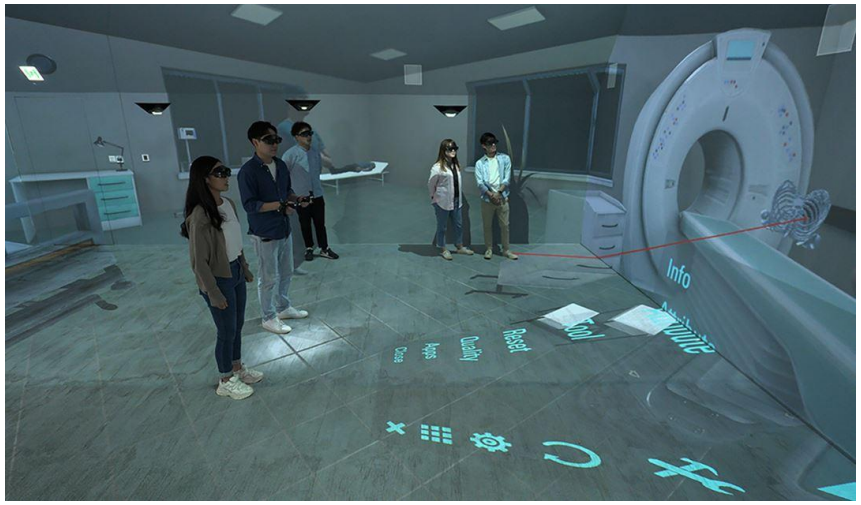
current XR headset display stack

Large & Specialty Displays

- Video walls: tiled LED panels driven as one logical framebuffer
- Projection mapping, CAVE rooms, digital signage

Unifying idea: wildly different scan-out hardware, same framebuffer abstraction

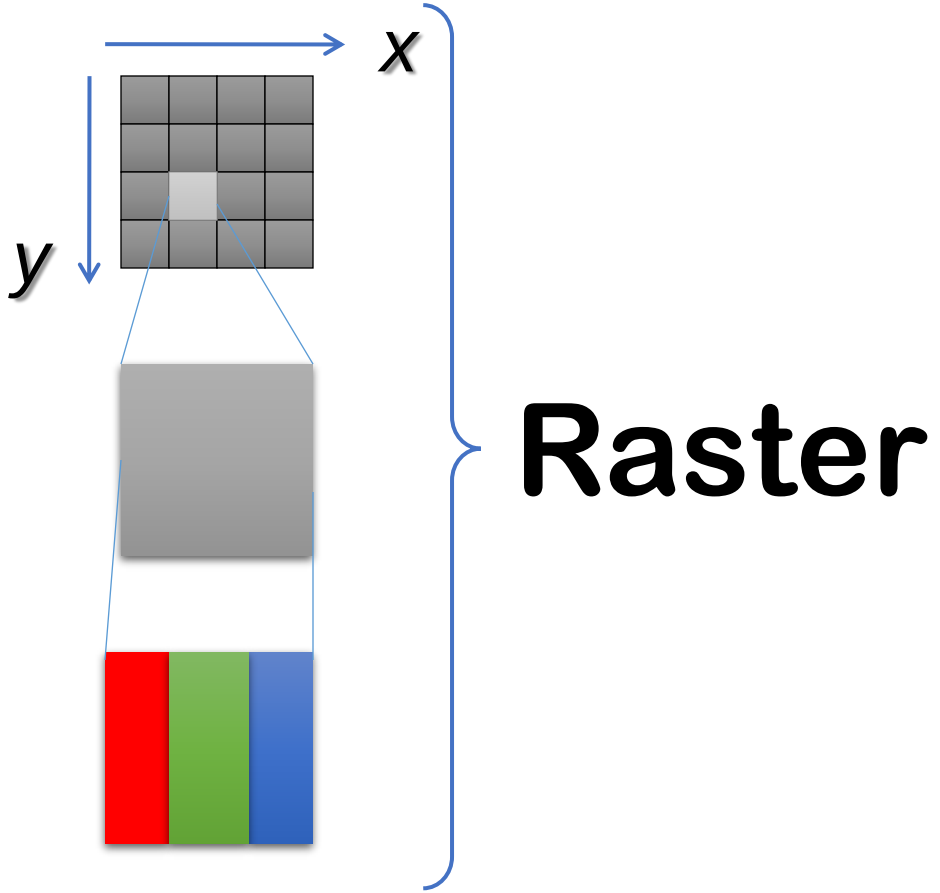


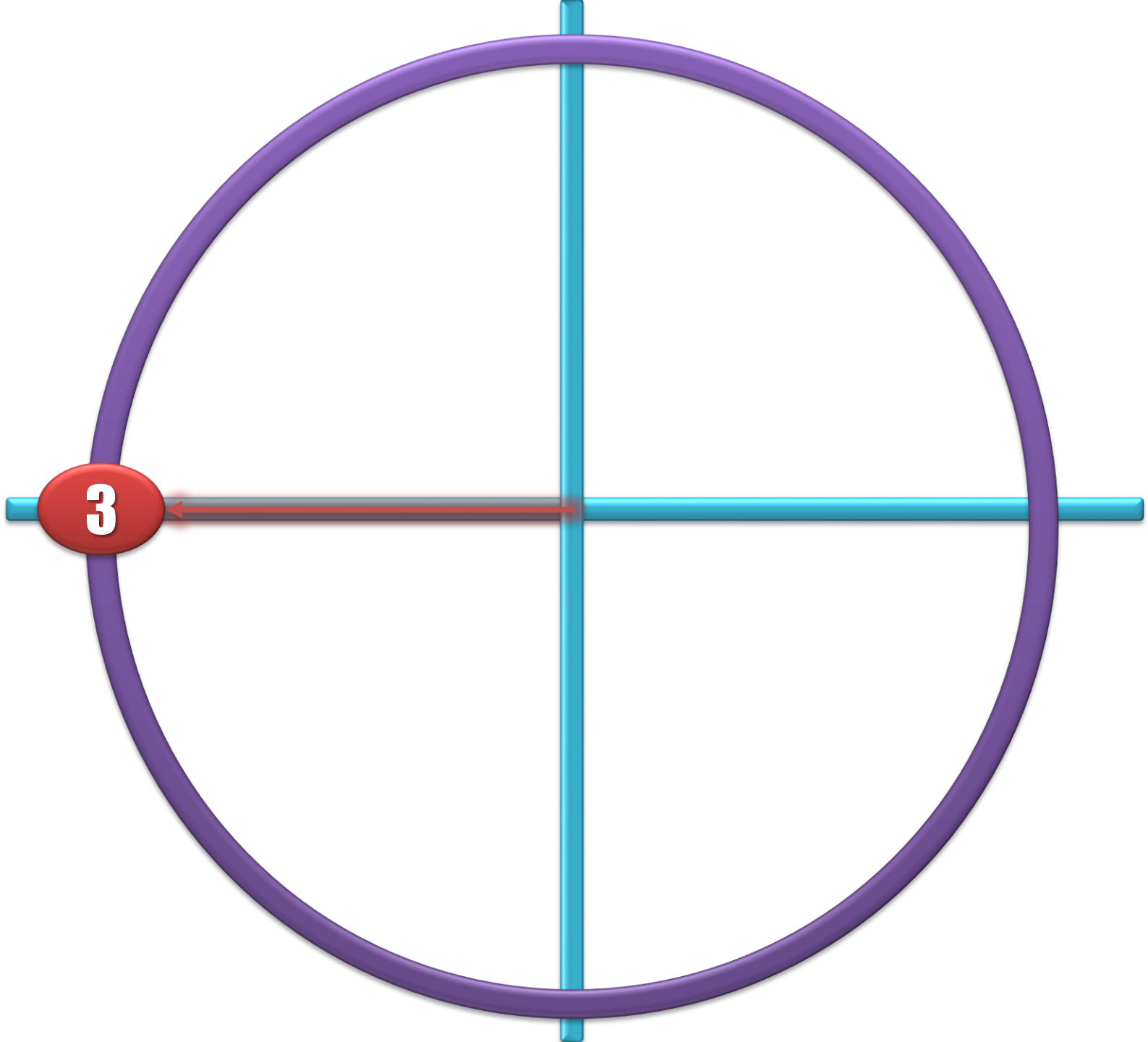


In Summary…

All FP Displays: Raster

- All flat panel displays have the same basic characteristics:
  - Active Matrix = address
  - Picture Element = Pixel
  - Sub-pixel elements: R, G, B




3

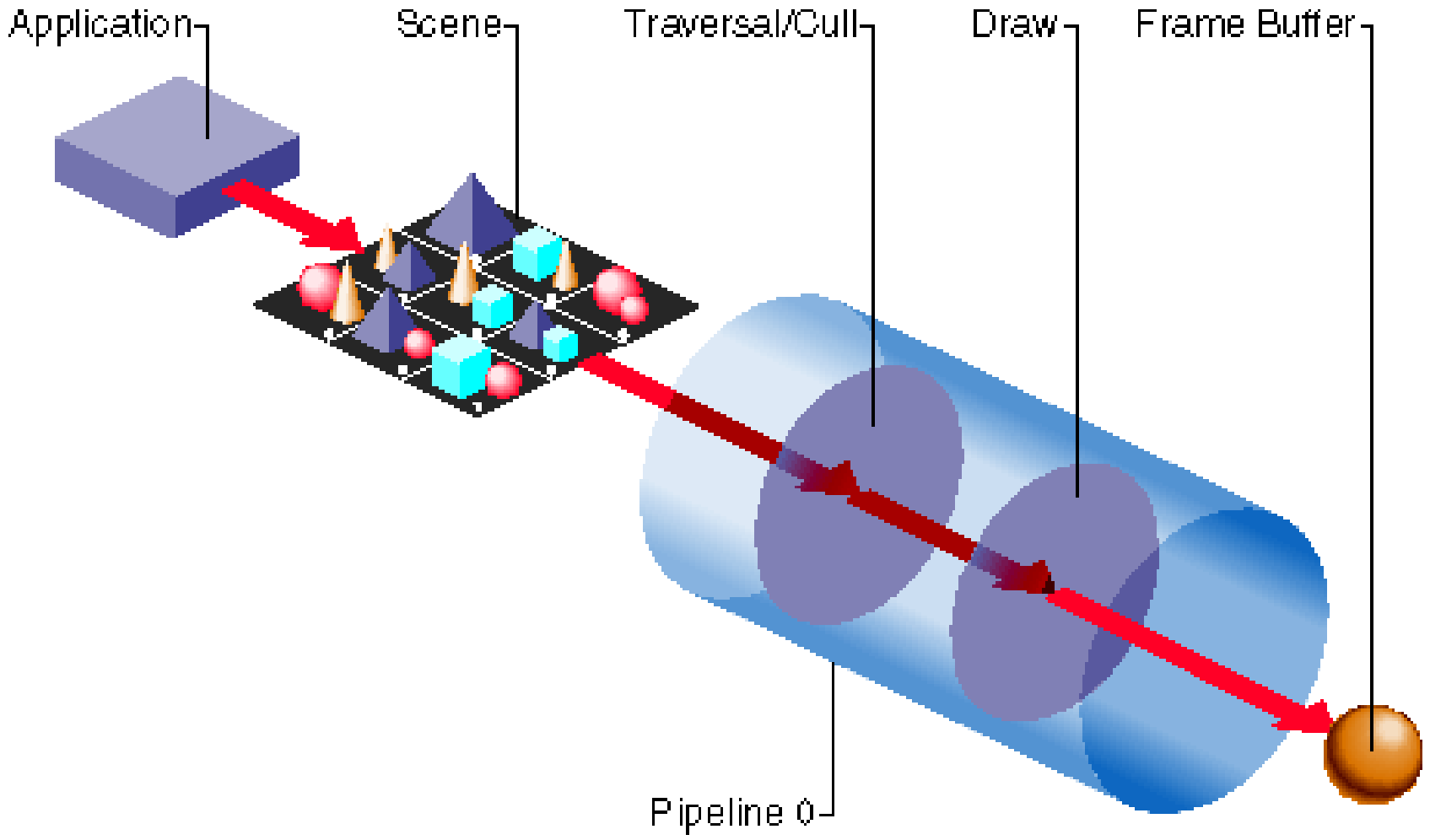
GPUs

The Rendering Pipeline

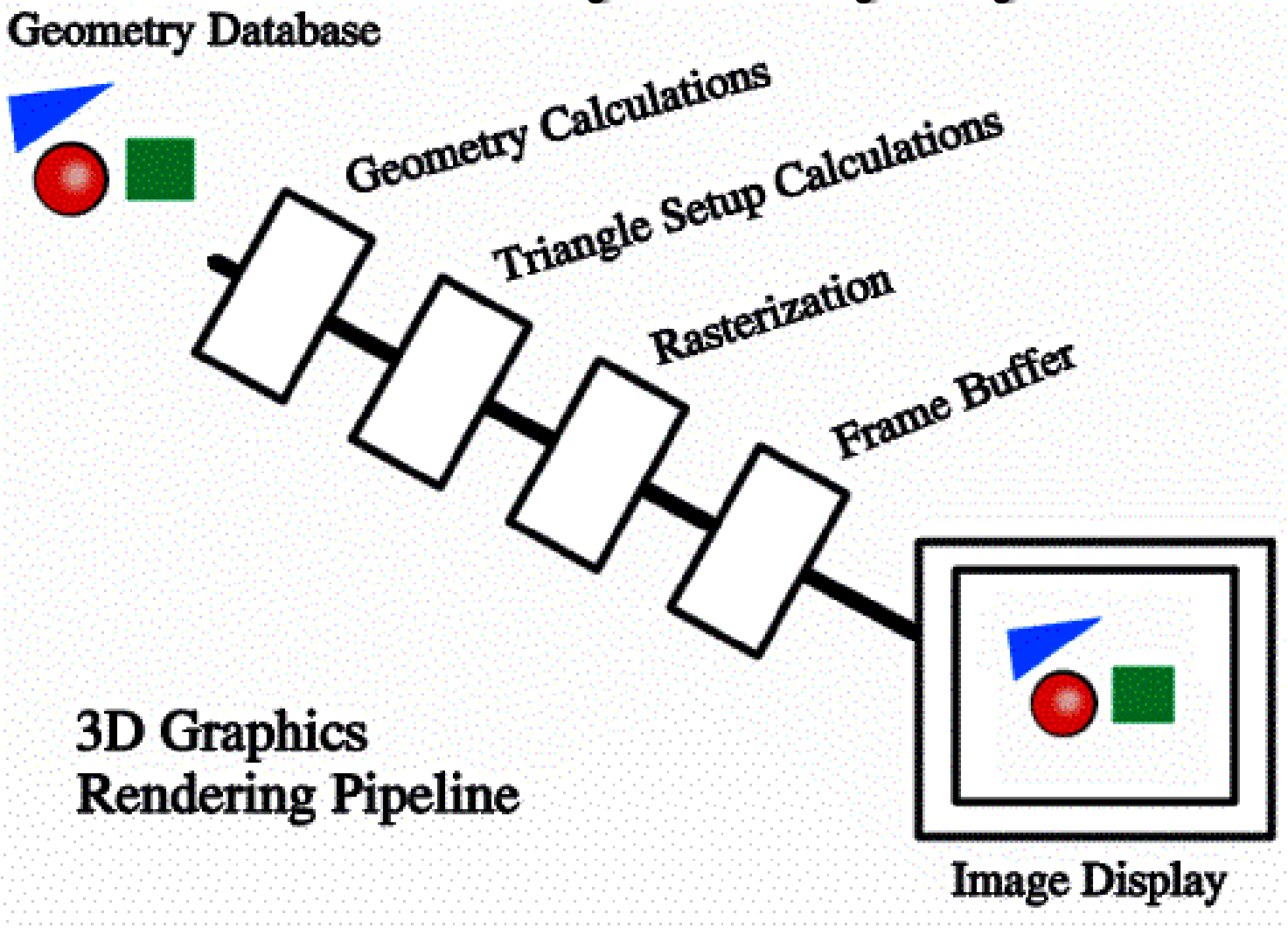
- Vertices in → transform → assemble → rasterize → shade → test & blend → framebuffer
- Fixed-function stages (rasterizer, blend) + programmable stages (vertex, fragment)

Two questions for this part: (1) WHY is a GPU built this way? (2) WHAT does that buy us?

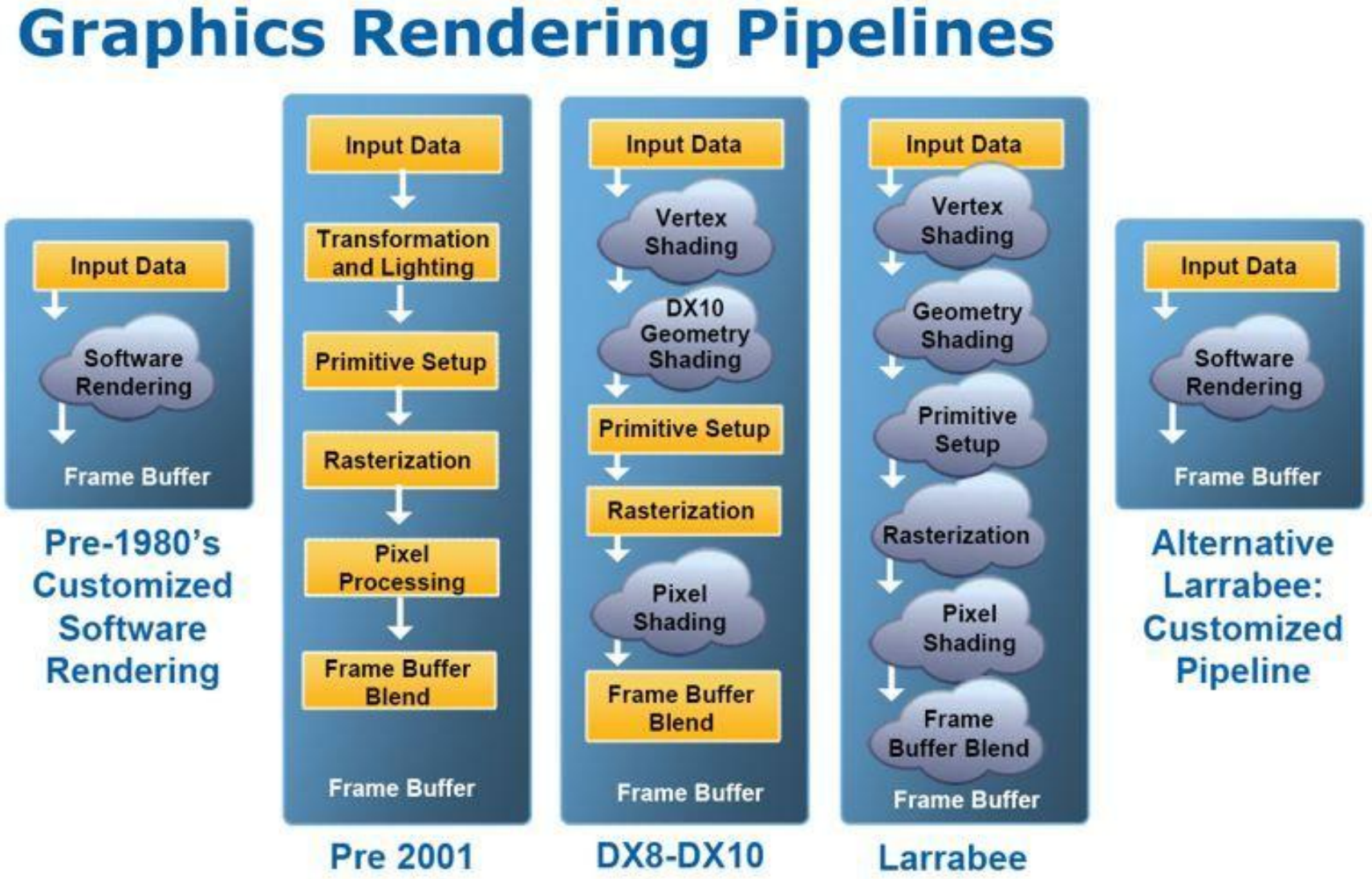
Rendering Pipeline



Inside a simple pipeline



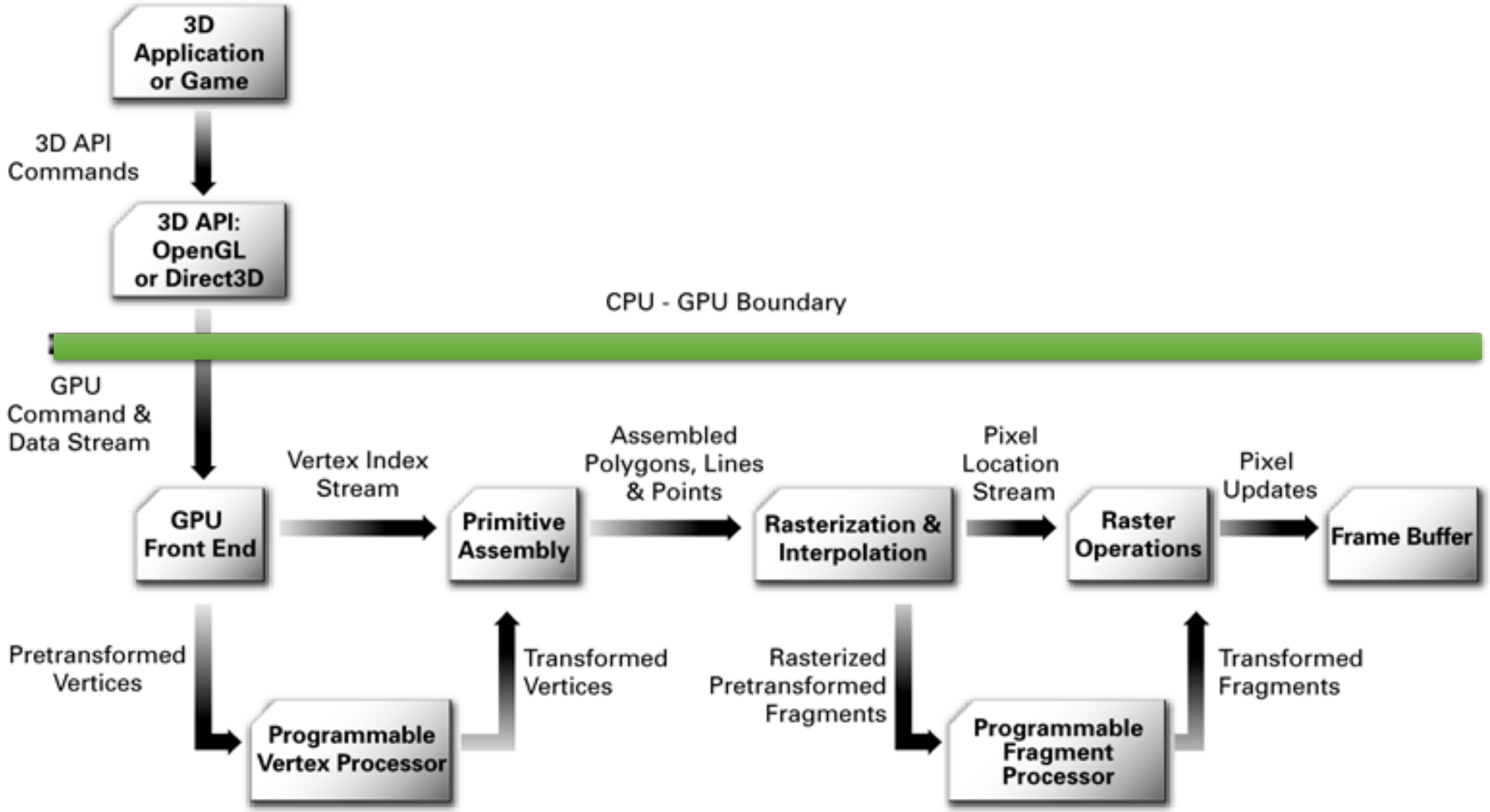
Short History



SGI

Sun Microsystems

CPU-GPU Boundary

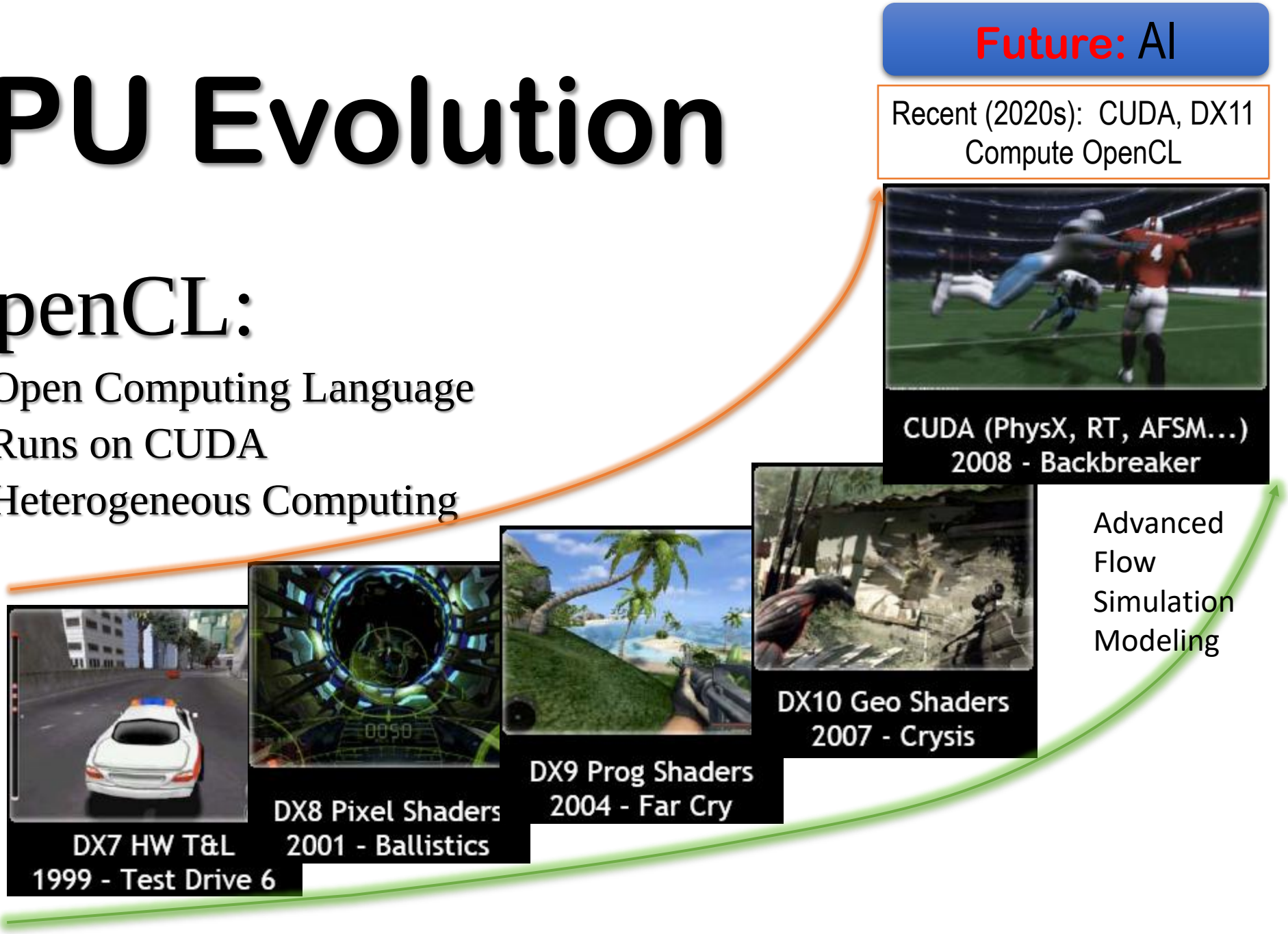


A Short History of Graphics Hardware

- 1980s-90s: SGI/Sun workstations — fixed-function pipes, one triangle at a time
- 2001: programmable vertex shaders (DirectX 8 era); 2002+: fragment shaders
- 2006: unified shader cores — one pool of cores runs any shader stage
- 2007: CUDA — GPUs open to general-purpose compute
- 2018: RT cores (ray tracing) + tensor cores (AI) become standard fixtures
- 2020s: AI in the frame itself (upscaling, frame generation, denoising)

GPU Evolution

- OpenCL:
  - Open Computing Language
  - Runs on CUDA
  - Heterogeneous Computing



Example GPU: NVIDIA Fermi

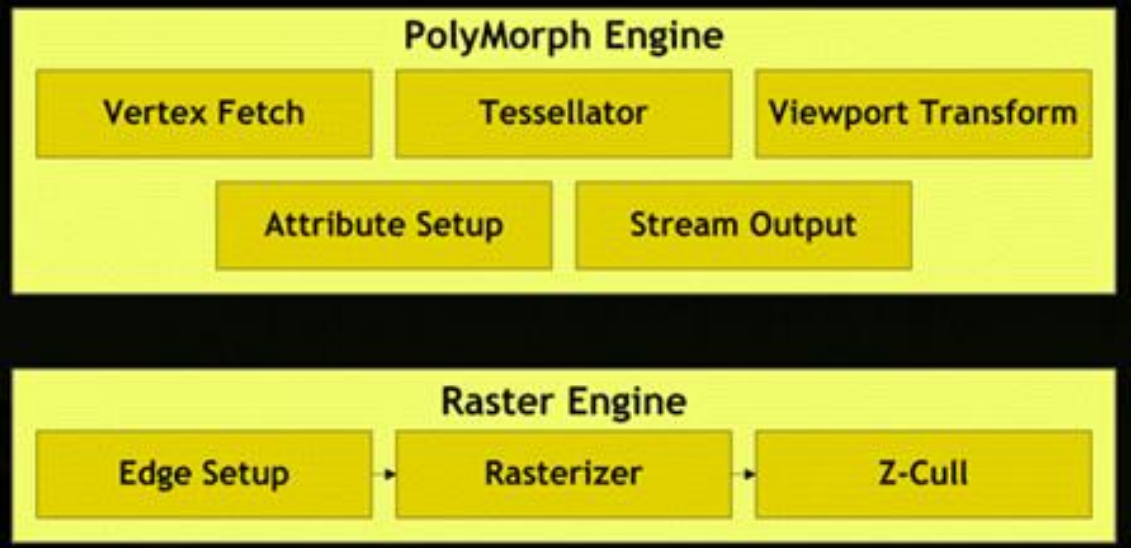
- NVIDIA Fermi GF100:
  - 512 CUDA stream processors
  - 16 Geometry Units
  - 4 Raster Units
  - 64 Texture Units,
  - 40nm fabrication process
  - 3.2 billion transistors
  - 700MHz core clock
  - 1.5GB GDDR5 memory
  - 384-bit memory interface
  - Up to 406W power consumption
  - 98° C



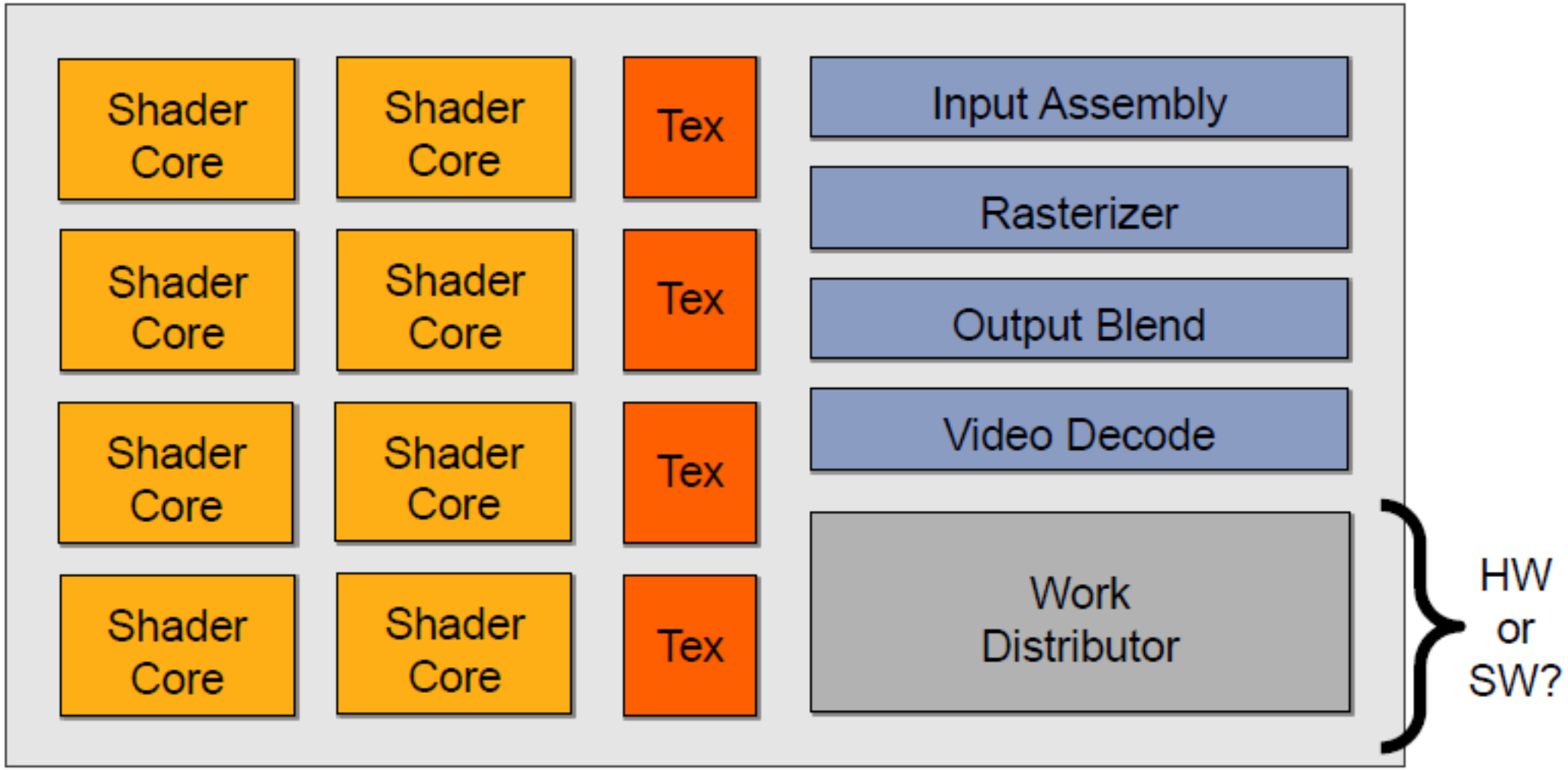
PolyMorph Engine

New GF100 PolyMorph and Raster Engines

- Dedicated graphics hardware
- PolyMorph Engine
  - World space processing
- Raster Engine
  - Screen space processing



GPUs Basics


Shader Core
Shader Core
Tex
Shader Core
Shader Core
Tex
Shader Core
Shader Core
Tex
Shader Core
Shader Core
Tex
Input Assembly
Rasterizer
Output Blend
Video Decode
Work Distributor
HW or SW?

Why Not Just a Faster CPU?

- Shading 2M pixels/frame: embarrassingly parallel — pixels don't talk to each other
- CPUs: few fat cores, huge caches, branch prediction — optimized for SERIAL latency
- GPUs: many small cores, narrow caches — optimized for THROUGHPUT

The rest of this lecture is one long answer to: how do you organize 20,000 simple cores?

Step 1

One Diffuse Shader

Will be explained in shader's chapter

- A diffuse reflectance shader: out = kd × max(0, n·l)
- Fragments processed independently — NO explicit parallelism in the shader code
- The parallelism is implicit: same program, millions of data items

A diffuse reflectance shader

```
sampler mySamp;
Texture2D<float3> myTex;
float3 lightDir;

float4 diffuseShader(float3 norm, float2 uv)
{
  float3 kd;
  kd = myTex.Sample(mySamp, uv);
  kd *= clamp( dot(lightDir, norm), 0.0, 1.0);
  return float4(kd, 1.0);
}
```

Shader programming model:

Fragments are processed independently

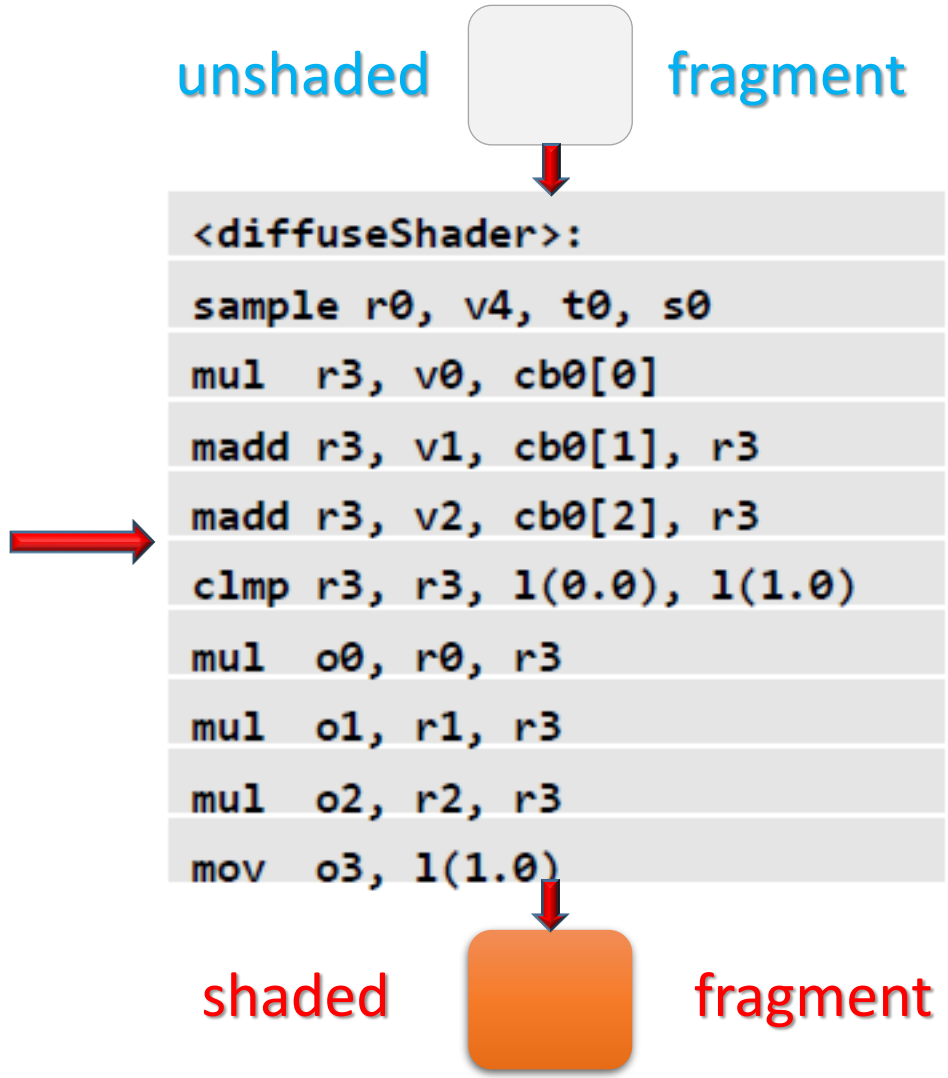
But, there is no explicit parallel programming

Step 2

Compile the shader program

```
sampler mySamp;
Texture2D<float3> myTex;
float3 lightDir;

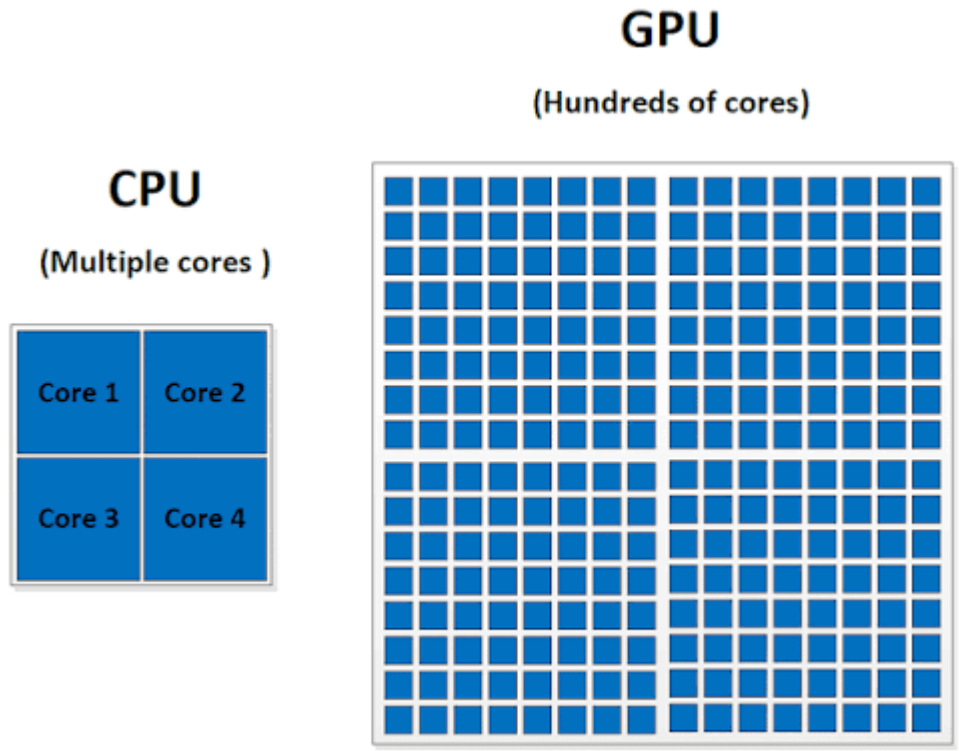
float4 diffuseShader(float3 norm, float2 uv)
{
  float3 kd;
  kd = myTex.Sample(mySamp, uv);
  kd *= clamp( dot(lightDir, norm), 0.0, 1.0);
  return float4(kd, 1.0);
}
```



Step 3

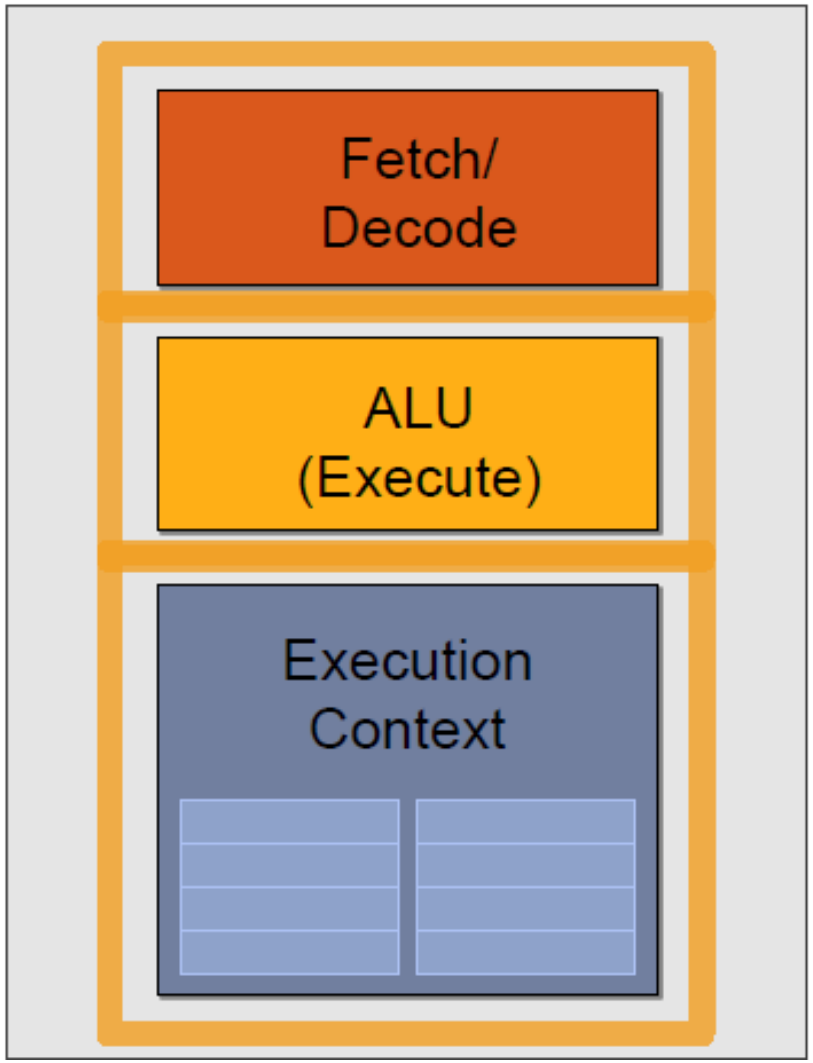
Execute on One (Slim) Core

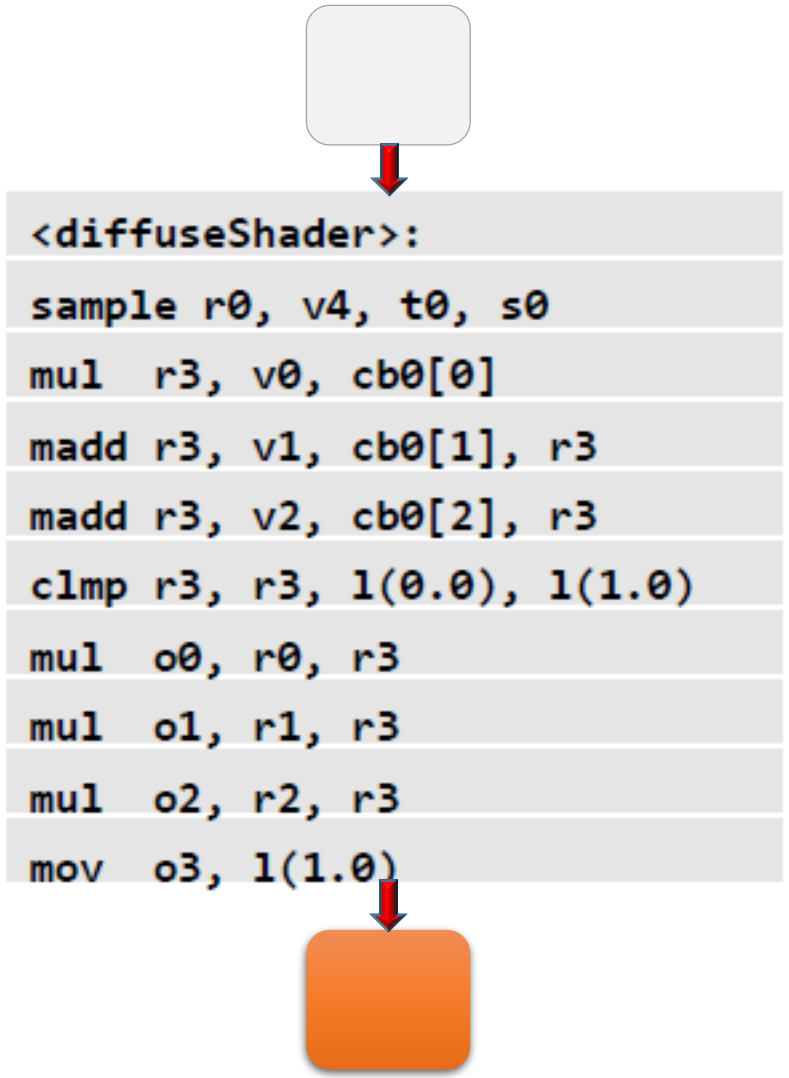
- Strip the CPU core: drop branch prediction, big cache, out-of-order engine
- Keep: ALU + registers + instruction fetch
- One slim core processes one fragment's instruction stream



fat CPU core vs slim GPU core

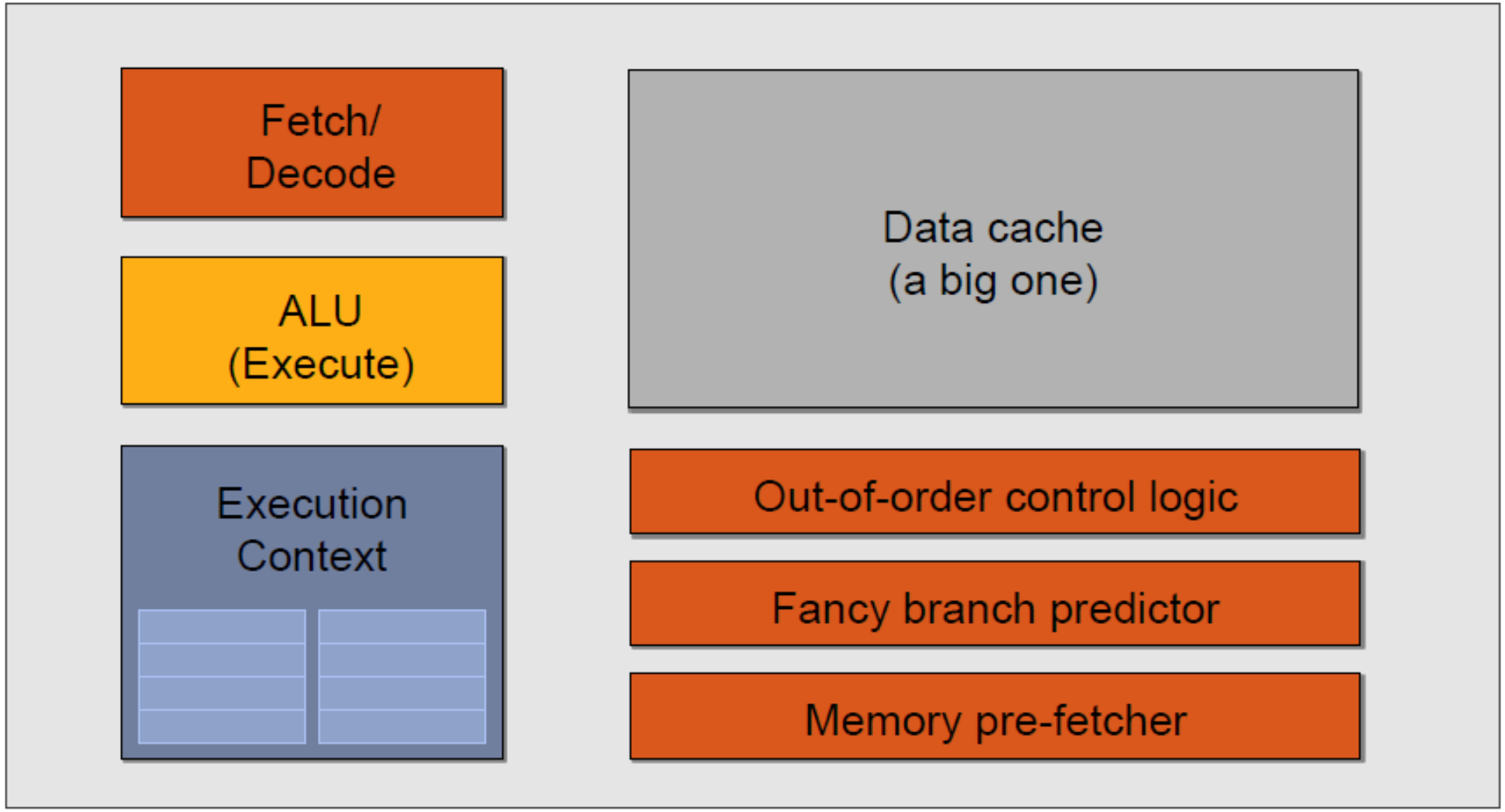
Execute the shader



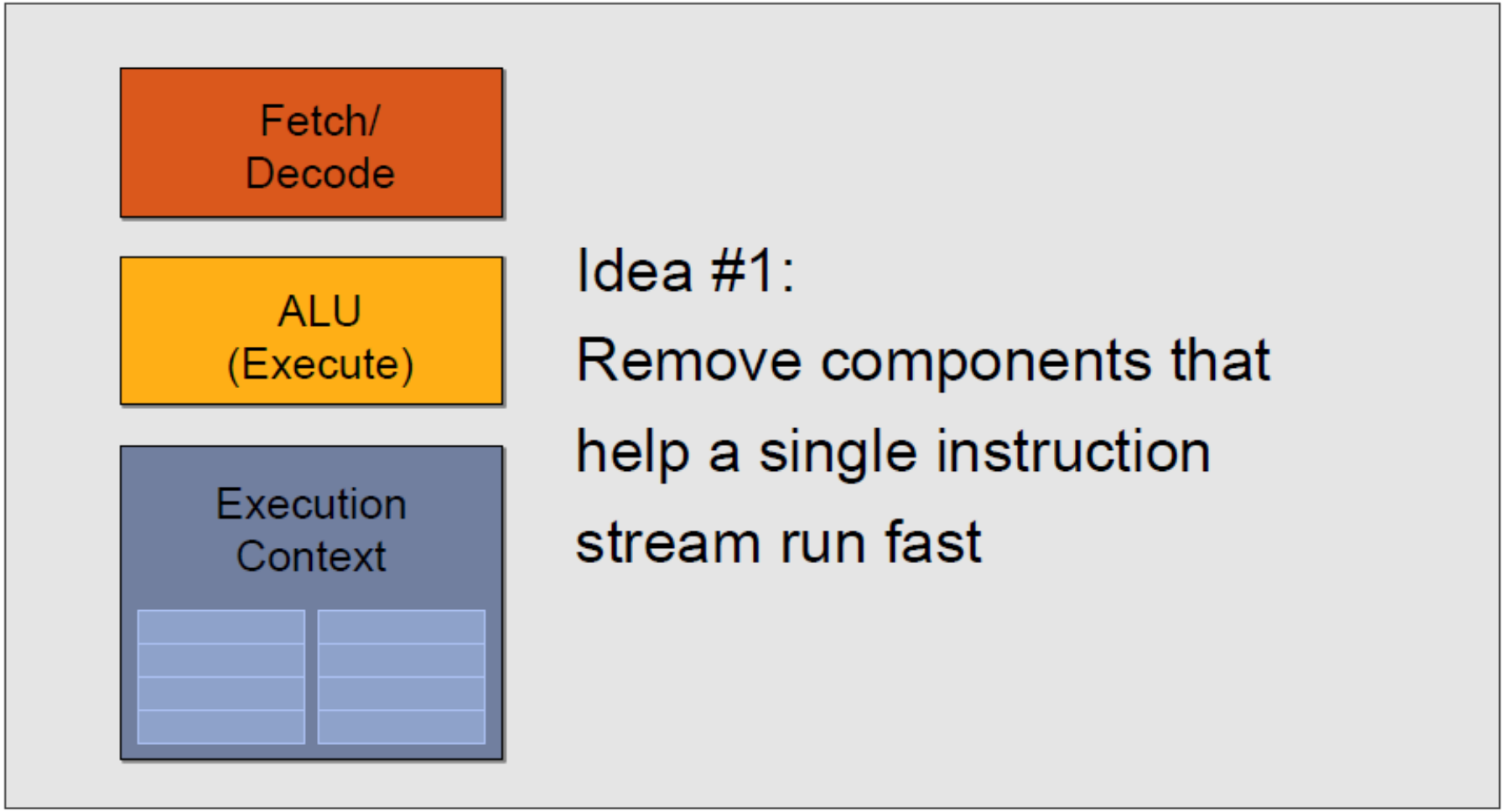


```
<diffuseShader>:
sample r0, v4, t0, s0
mul  r3, v0, cb0[0]
madd r3, v1, cb0[1], r3
madd r3, v2, cb0[2], r3
clmp r3, r3, l(0.0), l(1.0)
mul  o0, r0, r3
mul  o1, r1, r3
mul  o2, r2, r3
mov  o3, l(1.0)
```

CPU core style execution



Slimming down



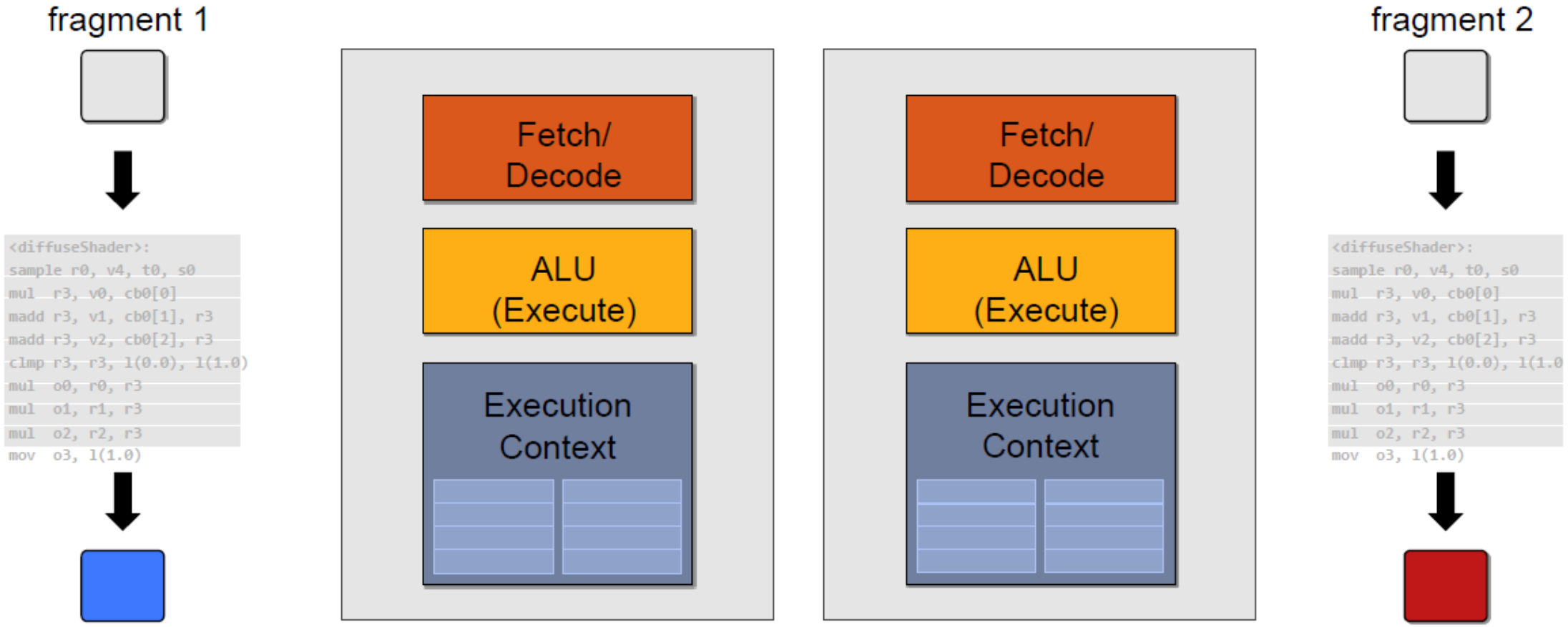
Step 4

Replicate: 2, 4, 16 Cores

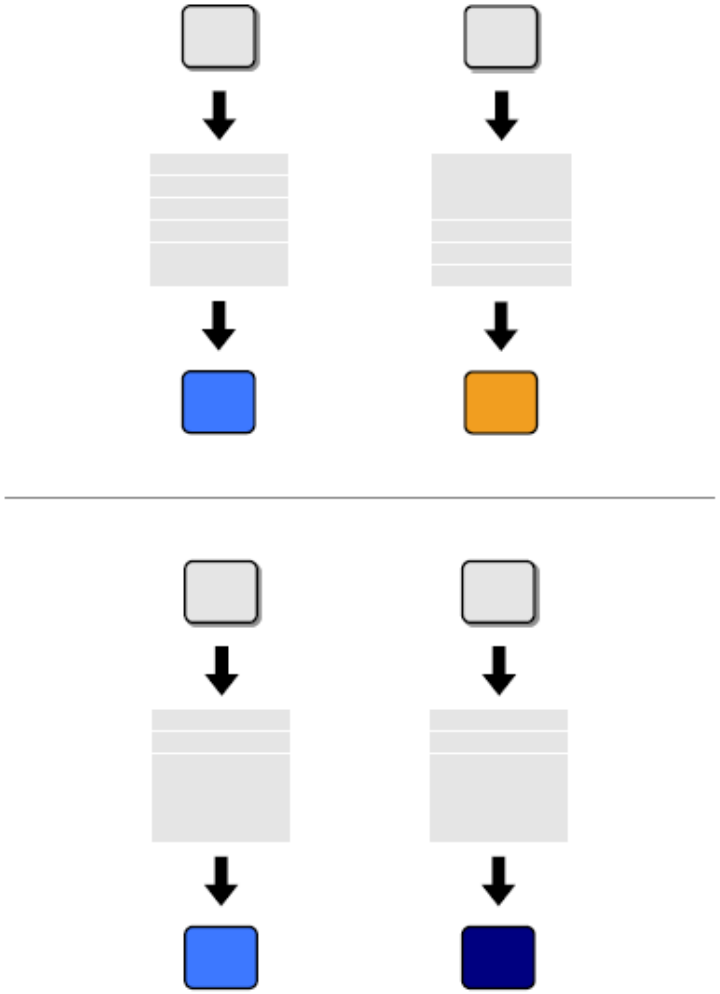
- 2 cores → 2 fragments in parallel; 4 → 4; 16 → 16
- But every core still fetches/decodes its own copy of the SAME instruction stream

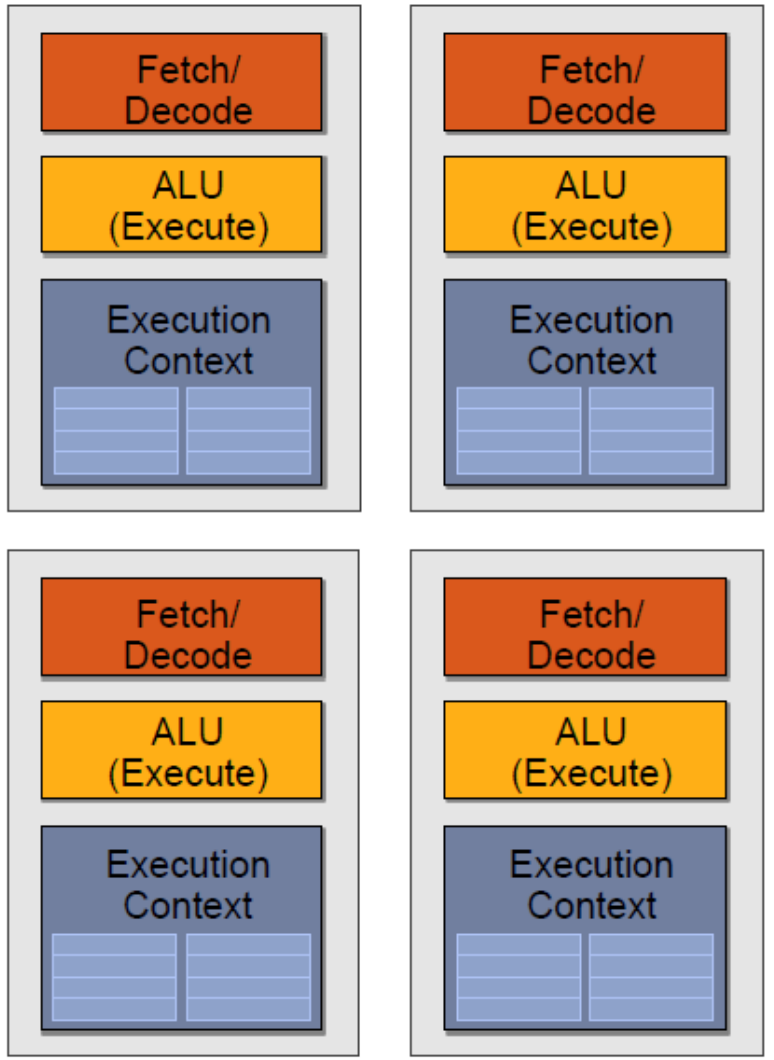
Waste alert: 16 fetch/decode units doing identical work

Two core – two fragments

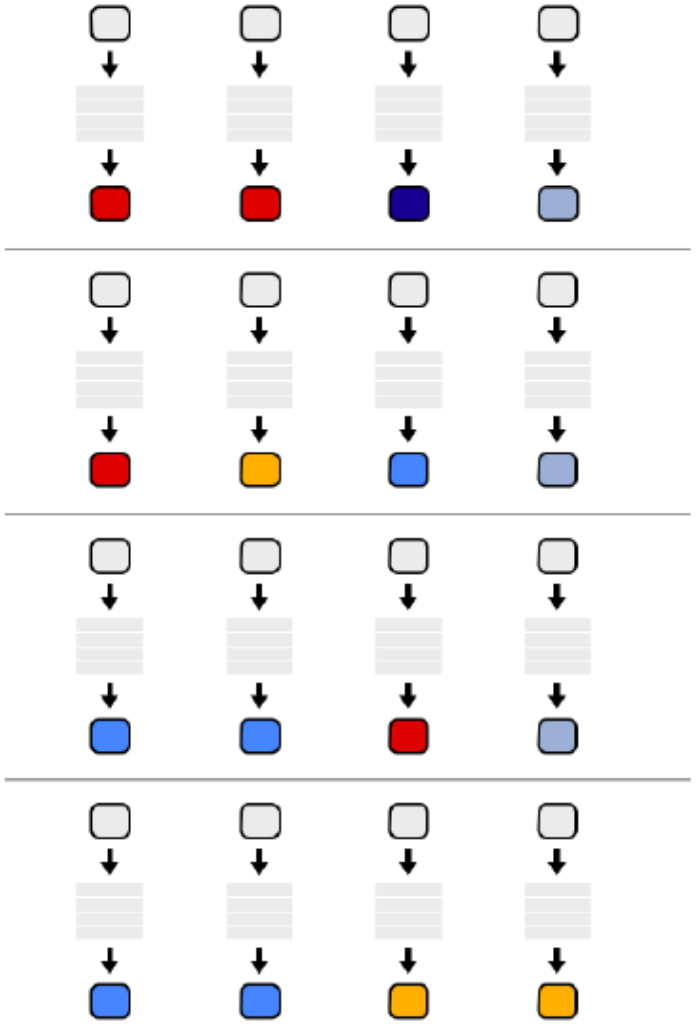


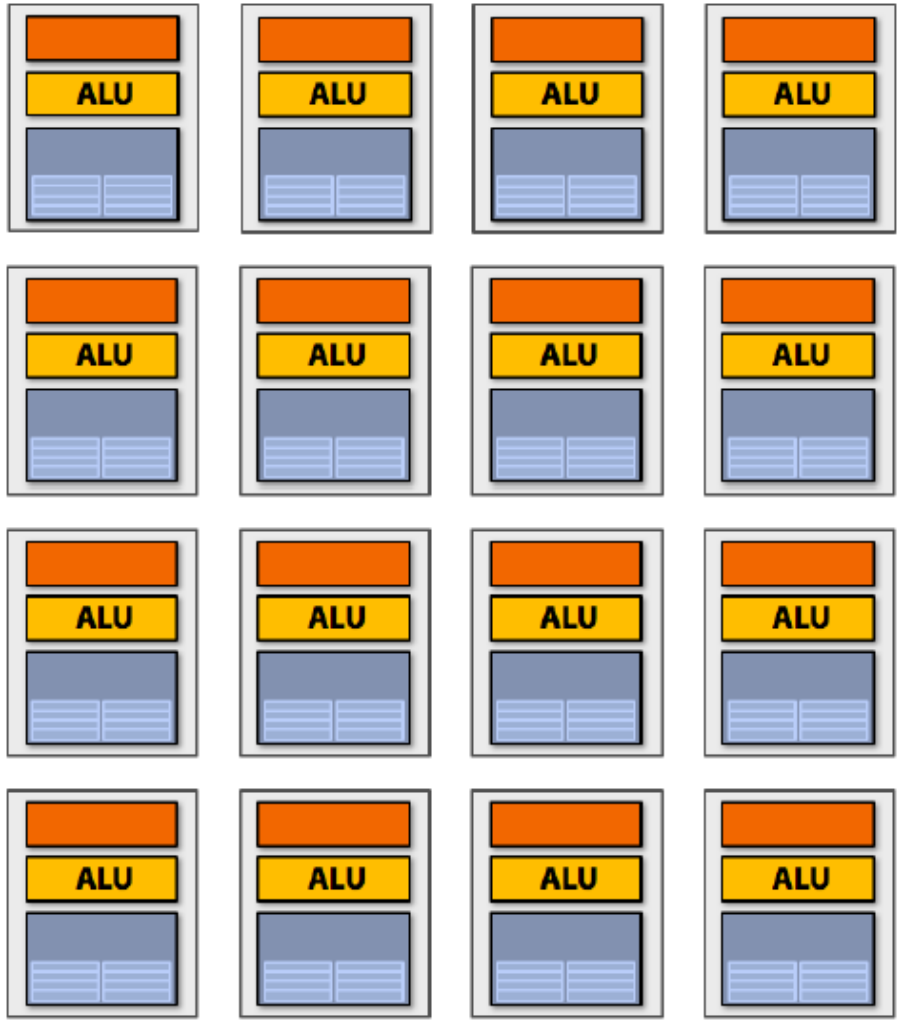
Four cores – Four fragments





16 cores – 16 fragments



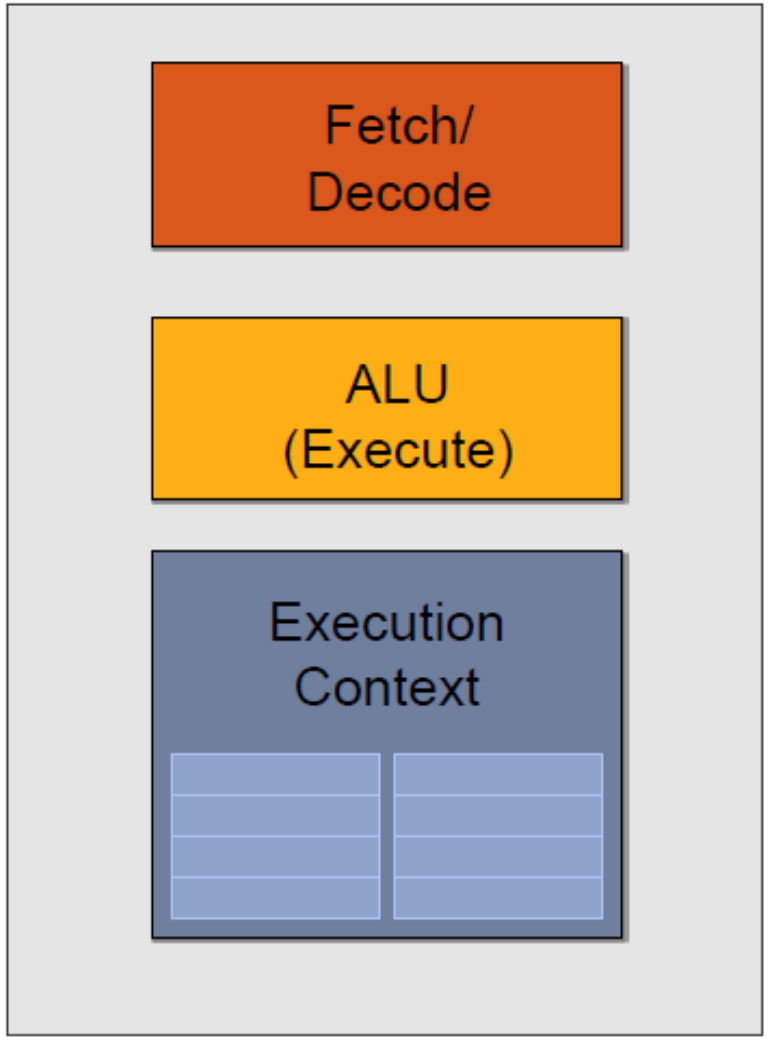


Step 5

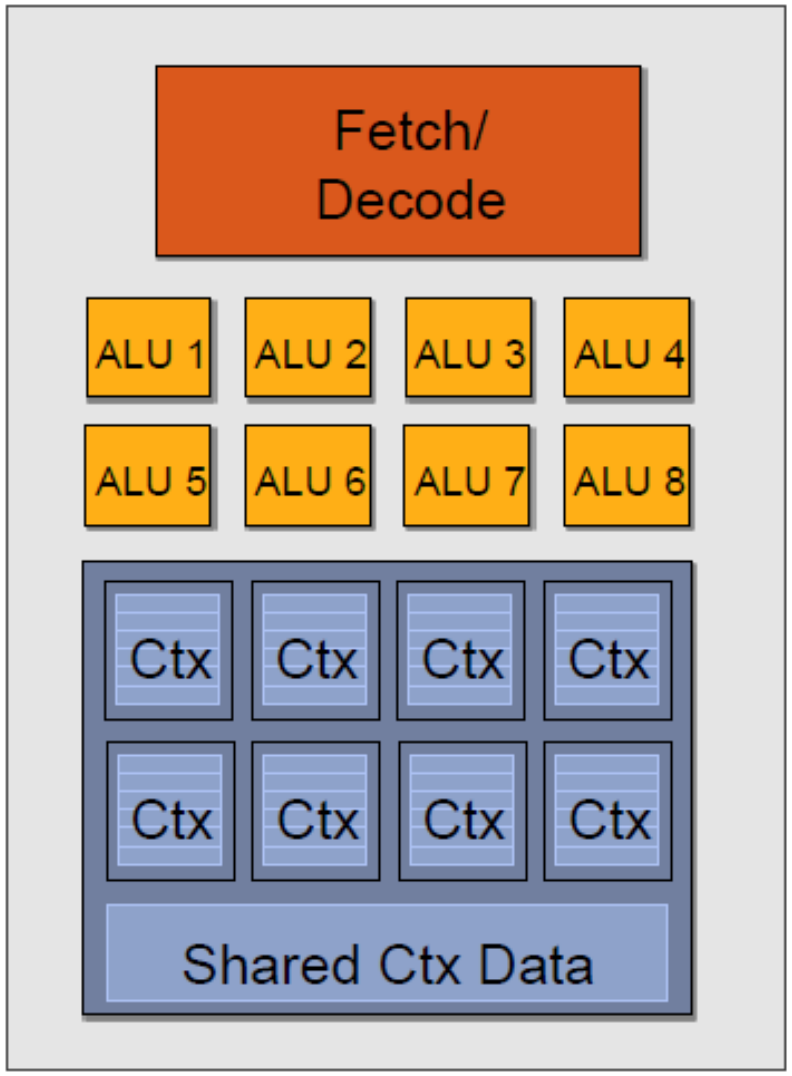
Share an instruction stream

- One fetch/decode unit drives MANY ALUs with the same instruction
- Many fragments should be able to share an instruction stream
- SIMD: Single Instruction, Multiple Data (GPU folk say SIMT — threads may diverge)
- Amortize the cost & complexity of instruction management across ALUs

A simple core



Add ALUs

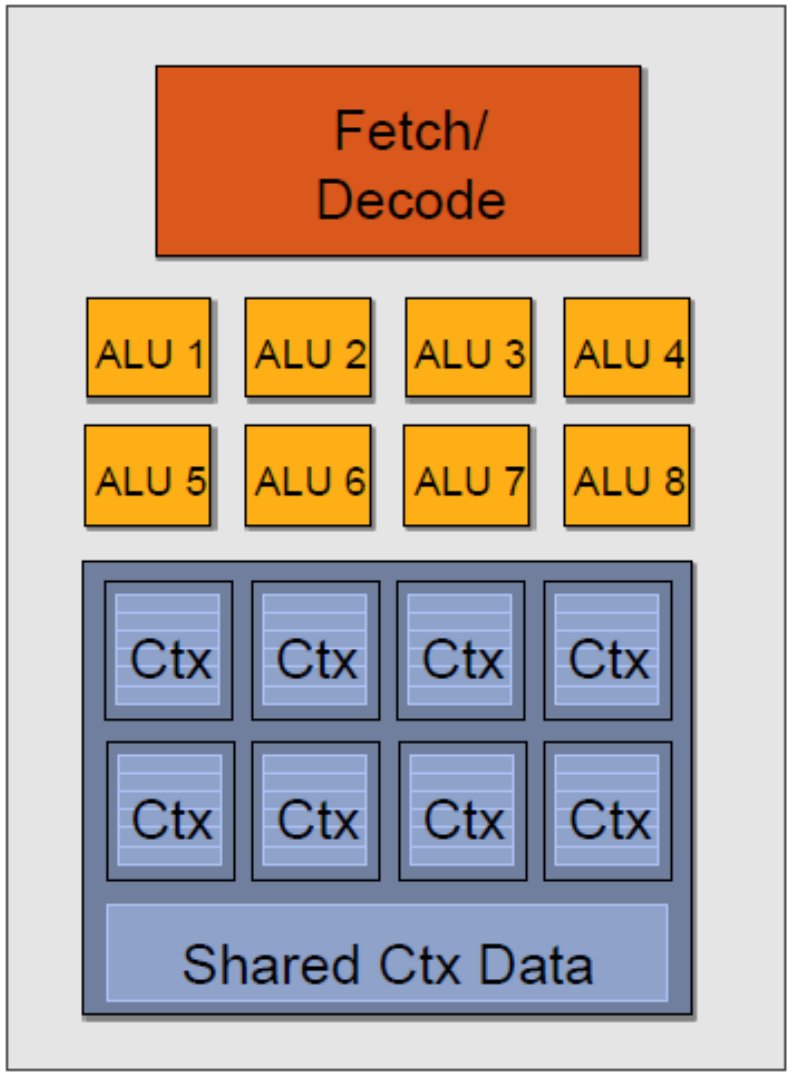


Amortize cost/complexity of managing an instruction stream across many ALUs

SIMD

Single Instruction Multiple Data

One Frag



Original Compiled Shader

```
<diffuseShader>:
sample r0, v4, t0, s0
mul  r3, v0, cb0[0]
madd r3, v1, cb0[1], r3
madd r3, v2, cb0[2], r3
clmp r3, r3, l(0.0), l(1.0)
mul  o0, r0, r3
mul  o1, r1, r3
mul  o2, r2, r3
mov  o3, l(1.0)
```

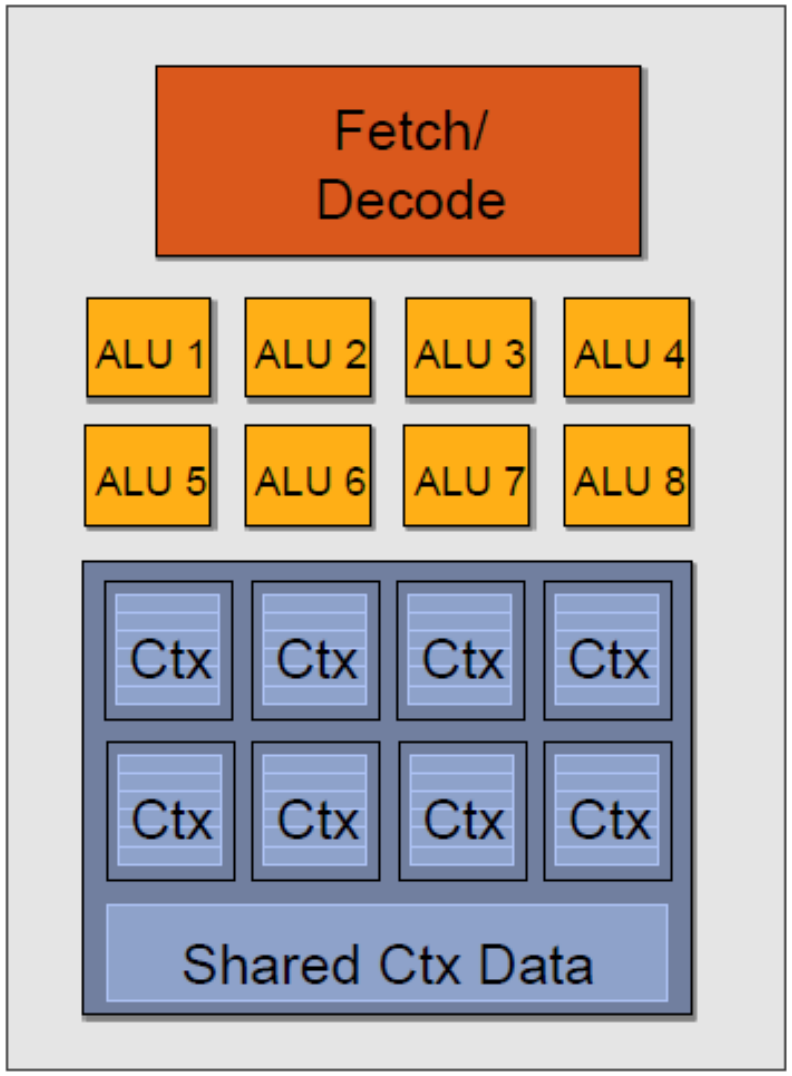
One fragment using scalar ops

Step 6

Vectorize the Shader

- Original compiled shader: scalar ops, 1 fragment
- Vectorized: 8-wide vector ops process 8 fragments per instruction
- Compiler's job (and hardware's) — your GLSL stays scalar & readable

Vectorize the Shader



New Compiled Shader

```
<VEC8_diffuseShader>:
VEC8_sample vec_r0, vec_v4, t0, vec_s0
VEC8_mul    vec_r3, vec_v0, cb0[0]
VEC8_madd   vec_r3, vec_v1, cb0[1], vec_r3
VEC8_madd   vec_r3, vec_v2, cb0[2], vec_r3
VEC8_clmp   vec_r3, vec_r3, 1(0.0), 1(1.0)
VEC8_mul    vec_o0, vec_r0, vec_r3
VEC8_mul    vec_o1, vec_r1, vec_r3
VEC8_mul    vec_o2, vec_r2, vec_r3
VEC8_mov    o3, 1(1.0)
```

8 fragments using vector ops

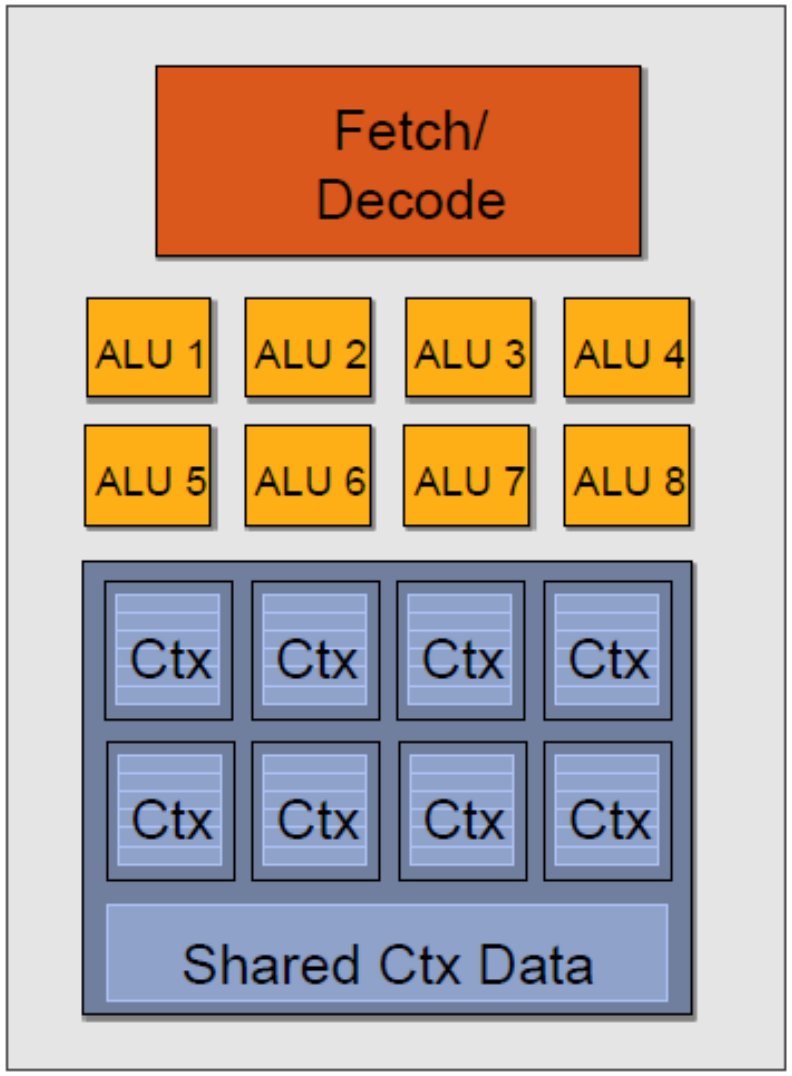
Step 5

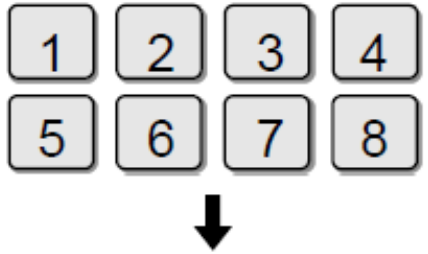
Scale Out: 8 → 128 Fragments

- 16 cores × 8 ALUs each = 128 fragments in flight
- Add latency hiding: while one group waits on memory, another executes
- Modern GPUs hold thousands of contexts to hide memory latency (no giant caches needed)

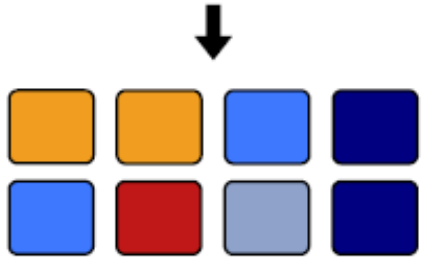
This is why GPUs want MANY small independent work items — and why your fragment shader shouldn't branch wildly

Multiple Frags

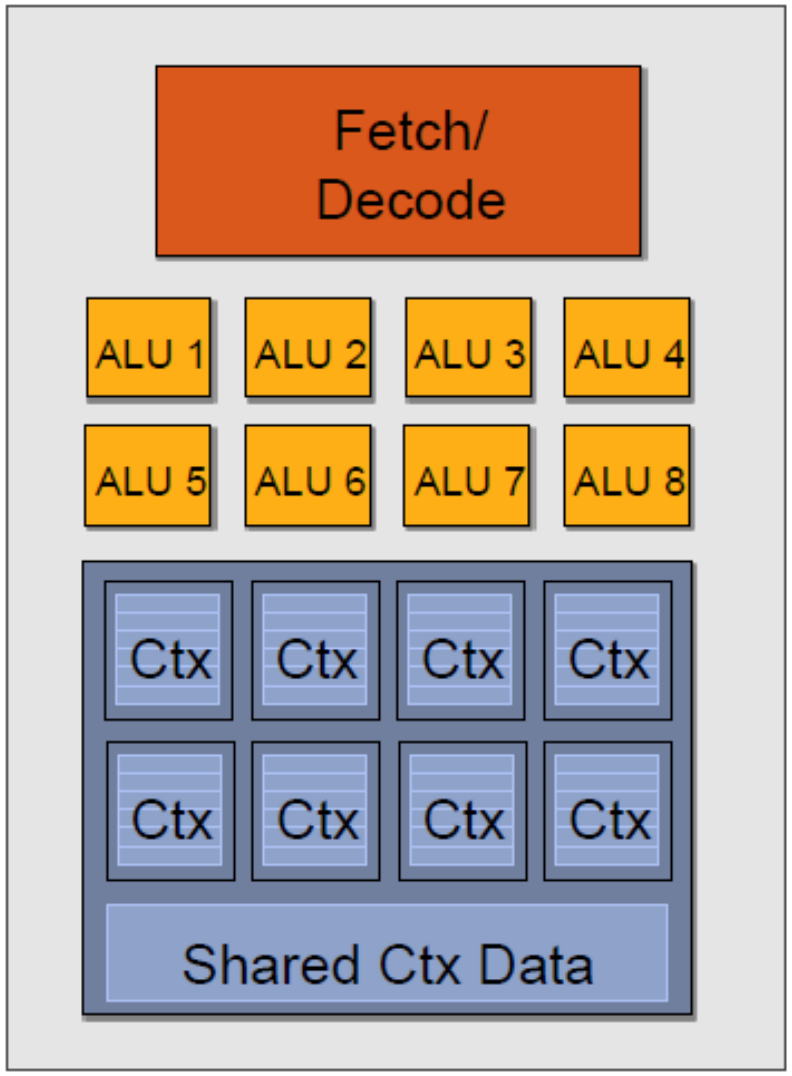


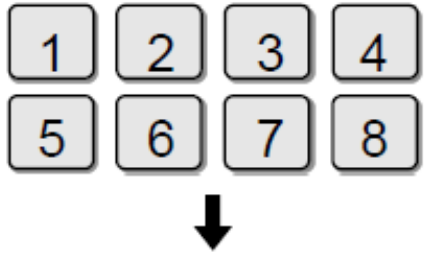



```
<VEC8_diffuseShader>:
VEC8_sample vec_r0, vec_v4, t0, vec_s0
VEC8_mul  vec_r3, vec_v0, cb0[0]
VEC8_madd vec_r3, vec_v1, cb0[1], vec_r3
VEC8_madd vec_r3, vec_v2, cb0[2], vec_r3
VEC8_clmp vec_r3, vec_r3, 1(0.0), 1(1.0)
VEC8_mul  vec_o0, vec_r0, vec_r3
VEC8_mul  vec_o1, vec_r1, vec_r3
VEC8_mul  vec_o2, vec_r2, vec_r3
VEC8_mov  o3, 1(1.0)
```

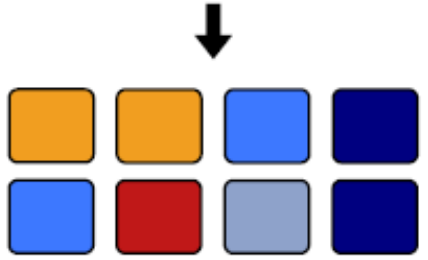


8 Fragments

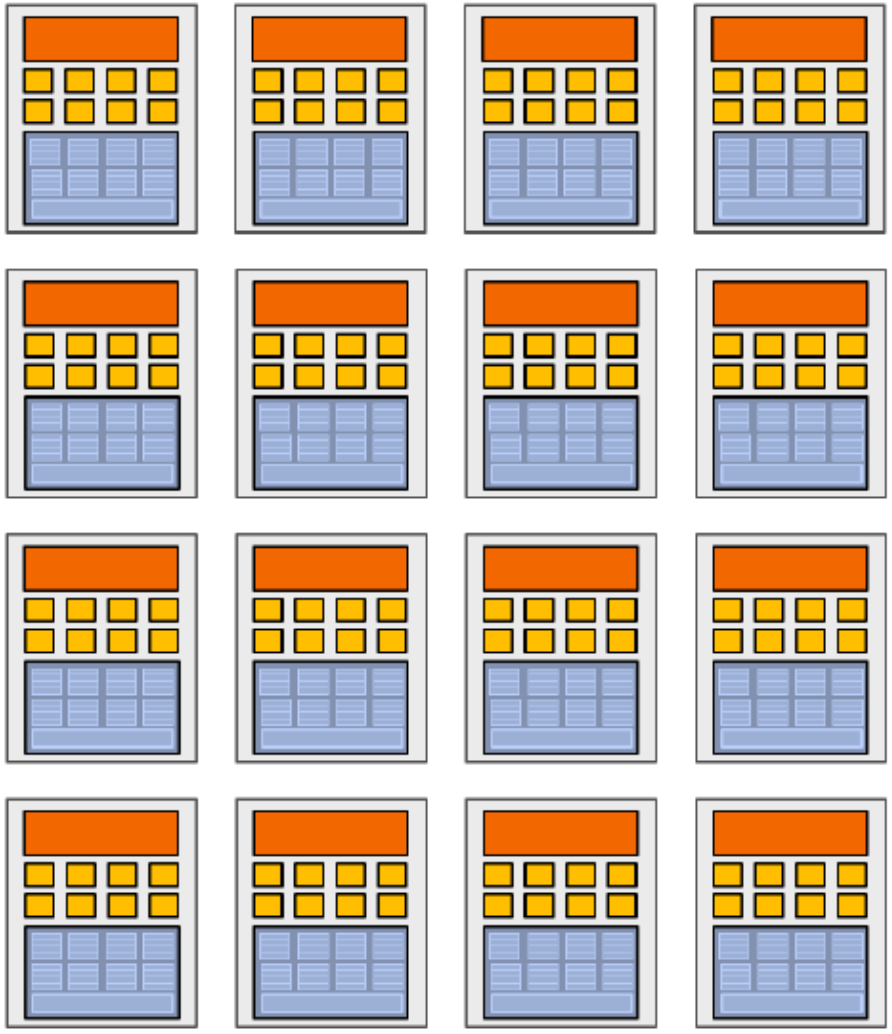





```
<VEC8_diffuseShader>:
VEC8_sample vec_r0, vec_v4, t0, vec_s0
VEC8_mul  vec_r3, vec_v0, cb0[0]
VEC8_madd vec_r3, vec_v1, cb0[1], vec_r3
VEC8_madd vec_r3, vec_v2, cb0[2], vec_r3
VEC8_clmp vec_r3, vec_r3, 1(0.0), 1(1.0)
VEC8_mul  vec_o0, vec_r0, vec_r3
VEC8_mul  vec_o1, vec_r1, vec_r3
VEC8_mul  vec_o2, vec_r2, vec_r3
VEC8_mov  o3, 1(1.0)
```

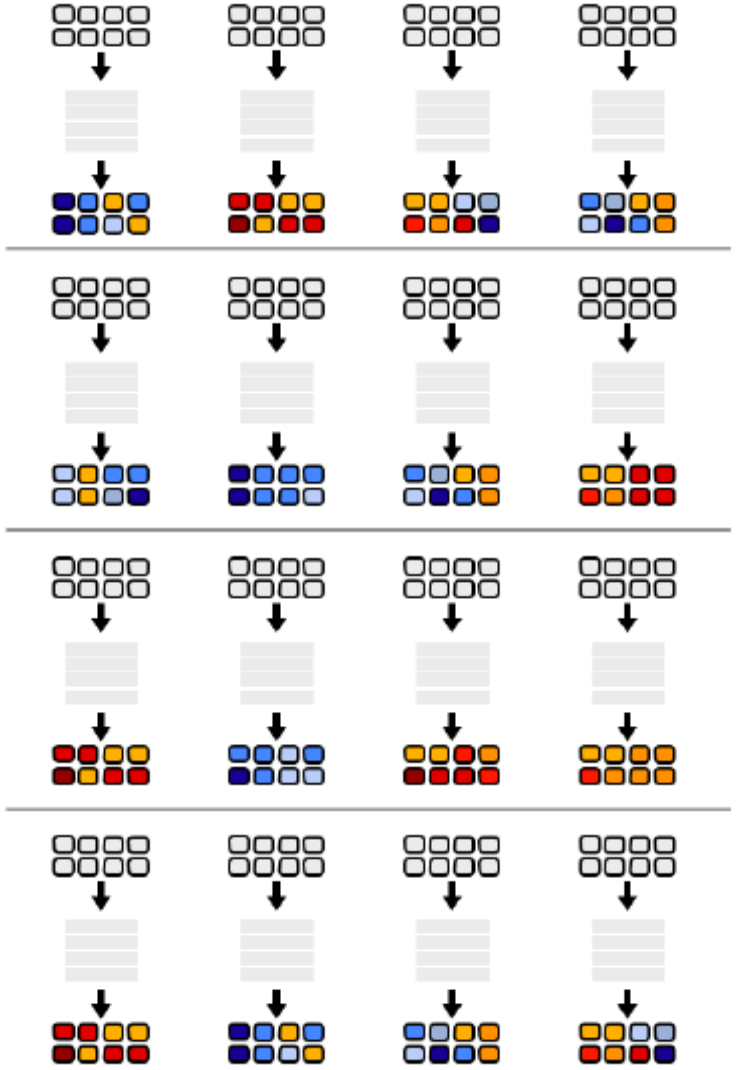


128 Fragments in II

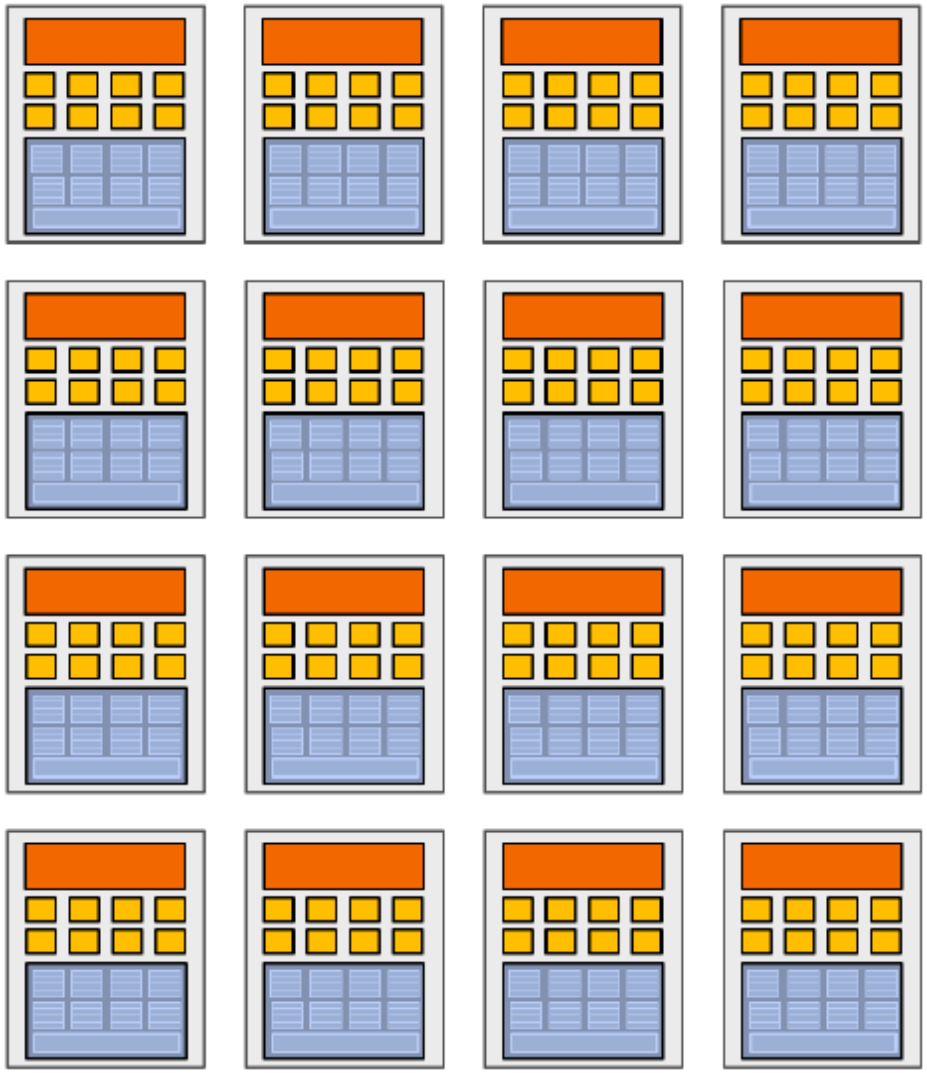


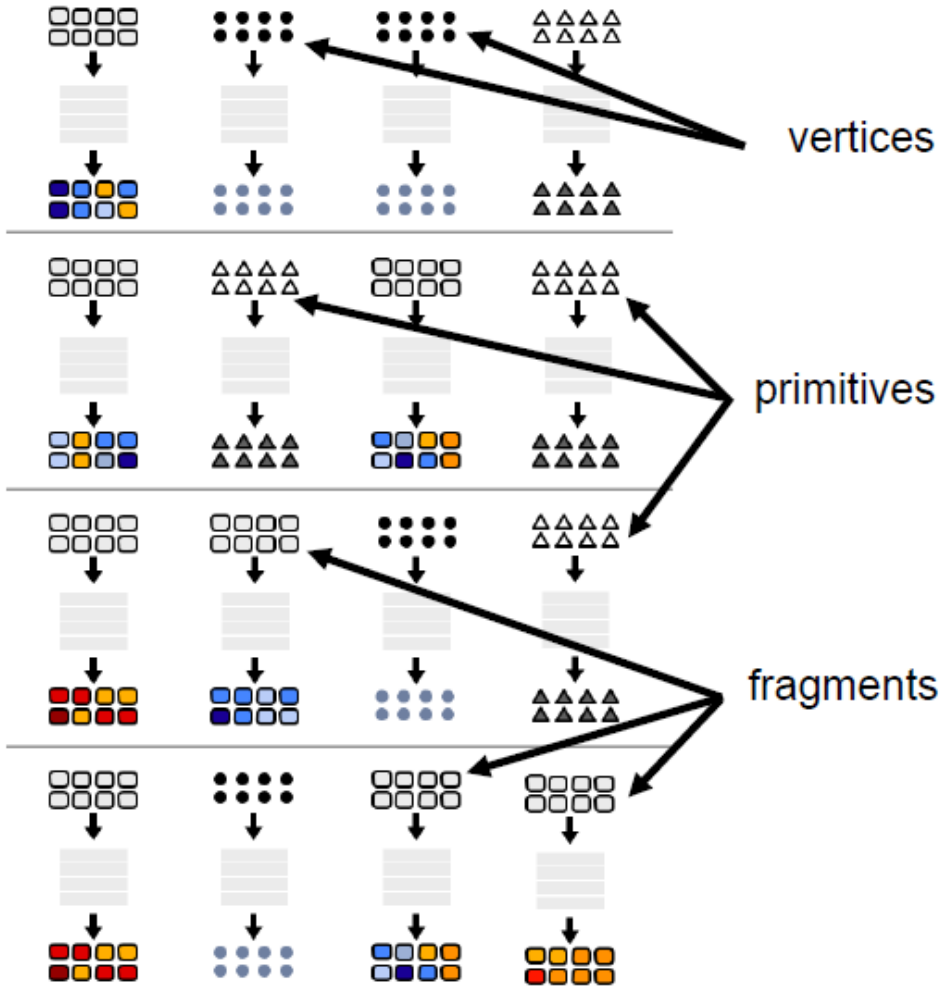
16 cores = 128 ALUs

16 instruction streams

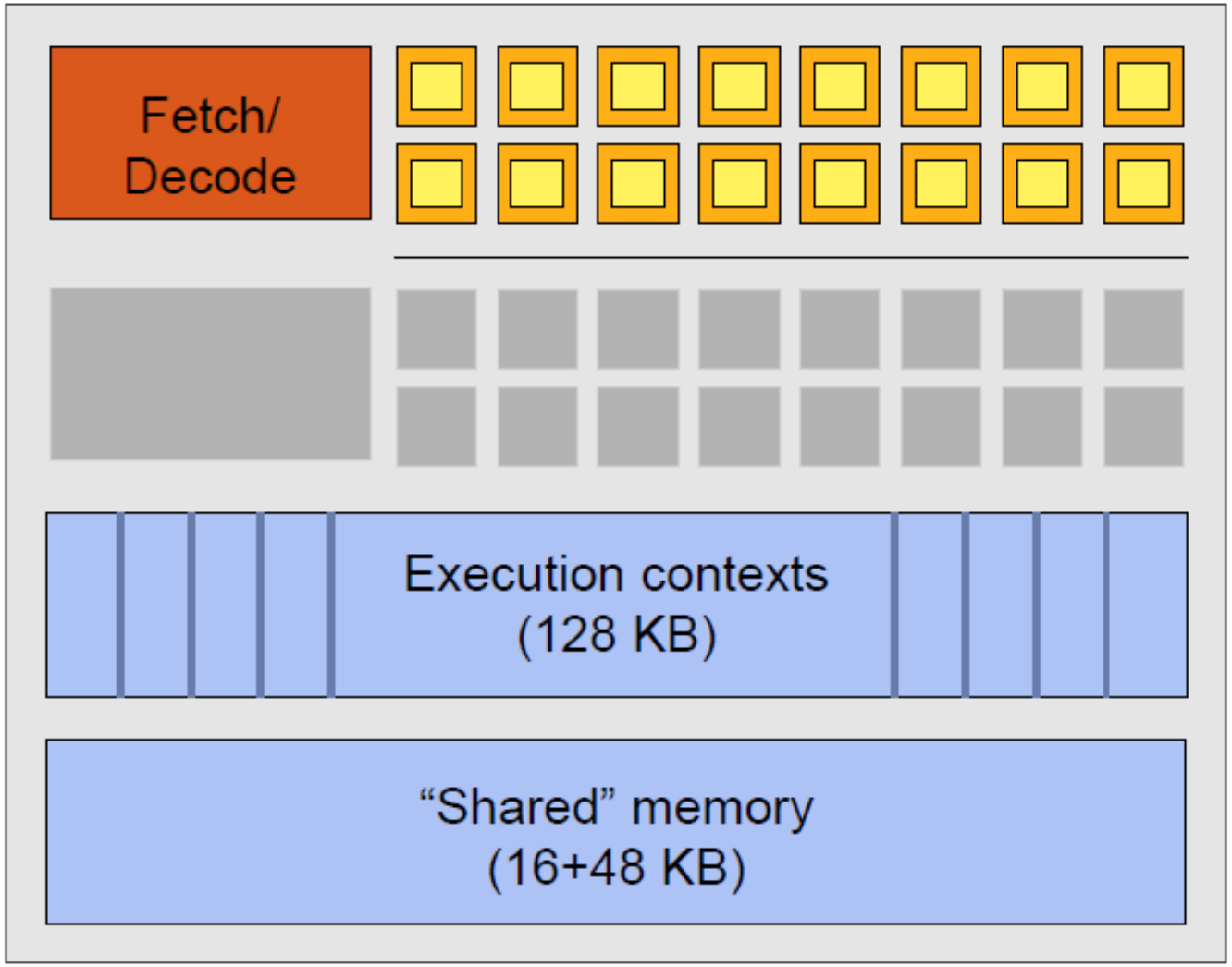


128 Vertices/Fragments Primitives





Example: NVIDIA GTX 580



Groups of 32 frags/vertices/CUDA threads share an instruction stream

Up to 48 groups are simultaneously interleaved

Up to 1536 individual contexts can be stored

Final Product - GTX 580



From Walkthrough to Silicon: the SM

- NVIDIA's building block: the Streaming Multiprocessor (SM)
- Blackwell SM: 4 processing slices × 32 CUDA cores = 128 cores/SM
- Each slice: warp scheduler + register file + SFU (sin, cos, rcp, √)
- 32 lanes = one 'warp' — literally the walkthrough's shared instruction stream

Modern Flagship: NVIDIA RTX 5090 (Blackwell, 2025)

- GB202 die: 92.2 billion transistors, TSMC 4NP, 575 W
- 170 SMs × 128 = 21,760 CUDA cores (the walkthrough's cores, at real scale)
- + 170 RT cores (BVH ray traversal) + 680 tensor cores (AI math)
- 32 GB GDDR7 on a 512-bit bus → 1,792 GB/s
- ~104 TFLOPS FP32 raster shader throughput; DLSS 4 multi-frame generation

Compare deck-of-old: Fermi (2010) had 3.2B transistors and 512 cores — 29× transistors, 42× cores in 15 years



The Other Two Engines: RT & Tensor Cores

- RT cores: hardware BVH intersection — rays × triangles without shader code
- Tensor cores: matrix engines for AI — denoising, upscaling (DLSS), frame generation
- Rasterization didn't disappear: hybrid rendering = raster + RT + AI in one frame

Modern Flagship II: Apple M4 & Unified Memory

- M4 (2024): 28B transistors, TSMC 3nm; 10-core GPU with hardware ray tracing & mesh shading
- Unified Memory Architecture: CPU, GPU, Neural Engine share ONE physical pool
- M4: 120 GB/s; M4 Pro: 273 GB/s; M4 Max: up to 546 GB/s, up to 128 GB shared

Figure 1 – Unified Memory Architecture

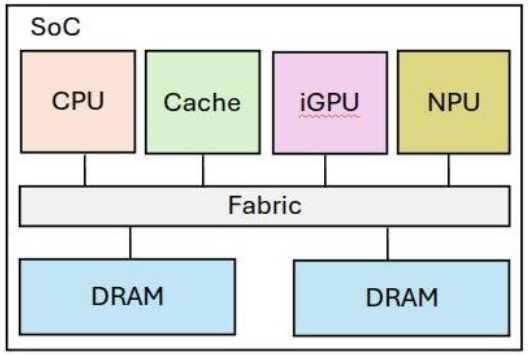
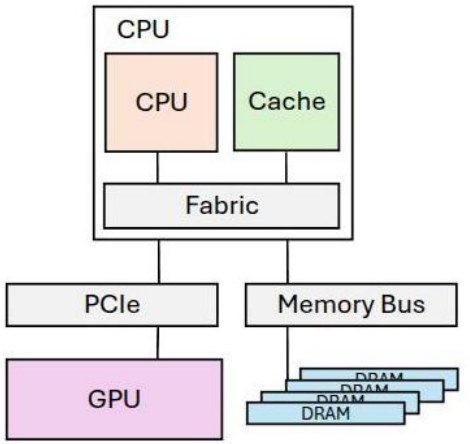
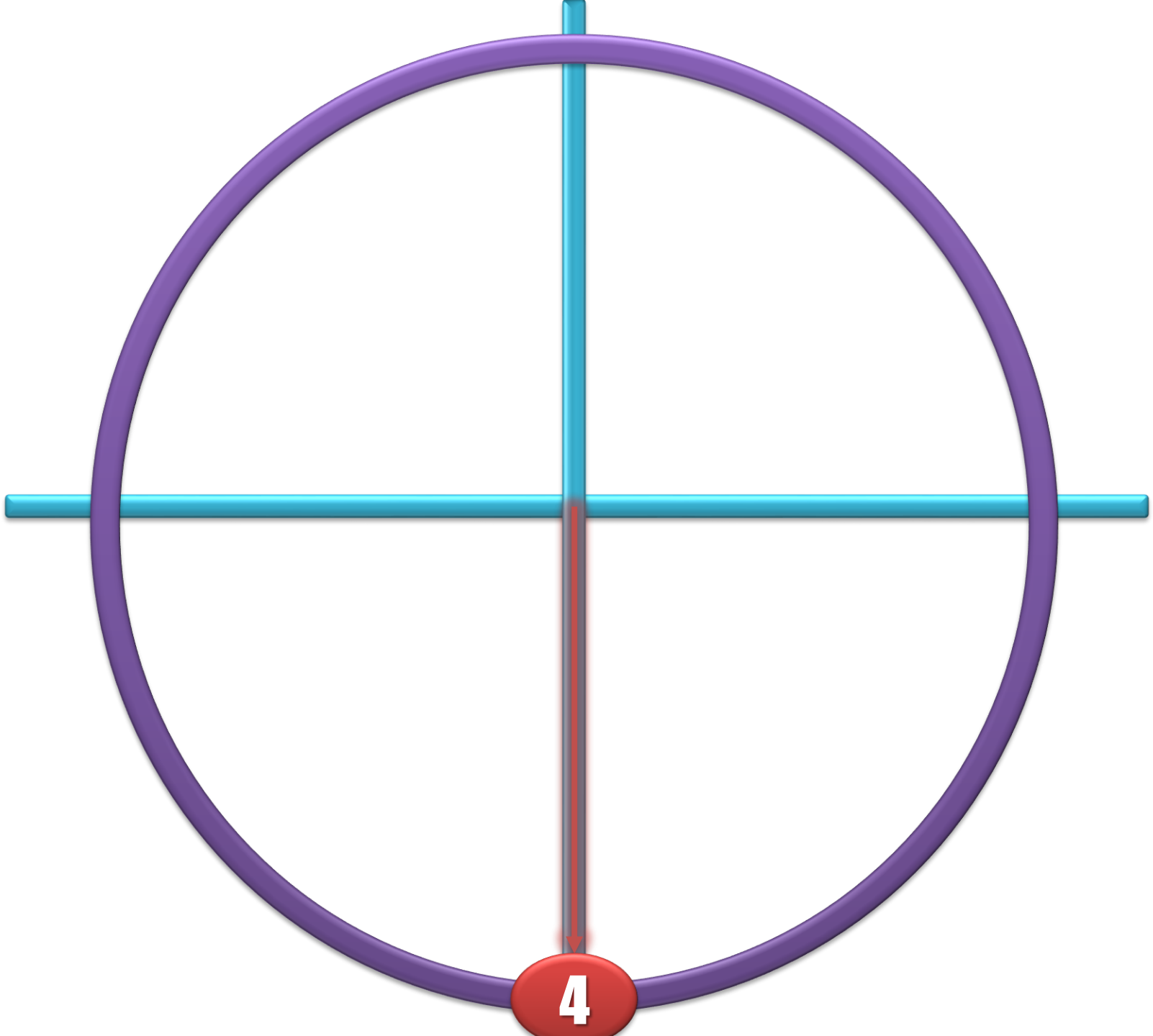


Figure 2 – Traditional Architecture



Consequence for graphics programmers: 'upload to VRAM' = copy over PCIe on discrete GPUs, but just a pointer on UMA — affects how you think about buffer uploads in labs


4

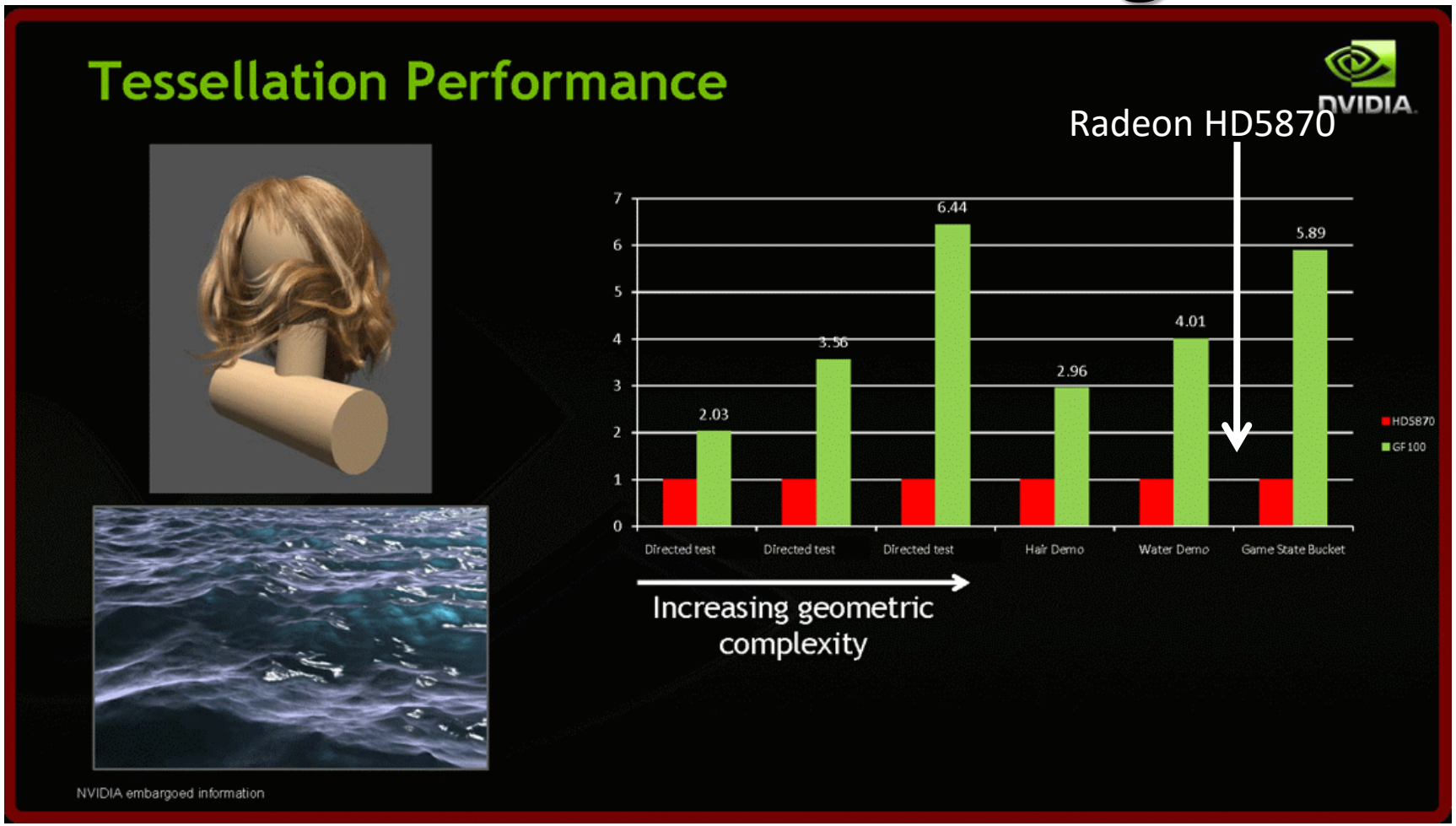
GPU Memory & Bandwidth: the Other Half of Performance

- Registers → L1/shared → L2 → VRAM: same pyramid idea as CPUs, different sizes
- GDDR7: ~1.8 TB/s on a 512-bit bus; HBM on datacenter parts: several TB/s
- Fragment shaders are often BANDWIDTH-bound, not math-bound

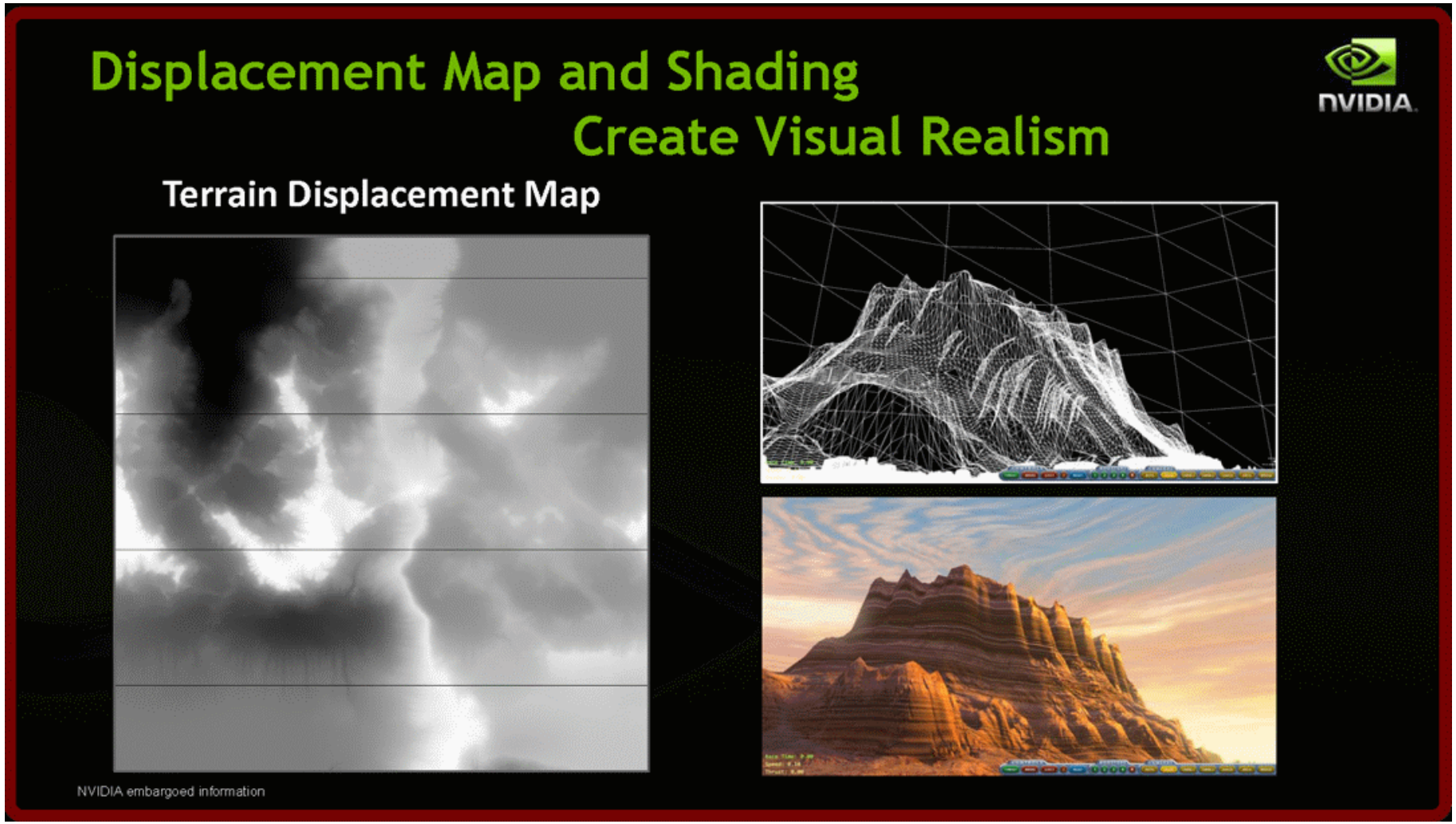
Performance

Current top of the Line: Radeon HD5870

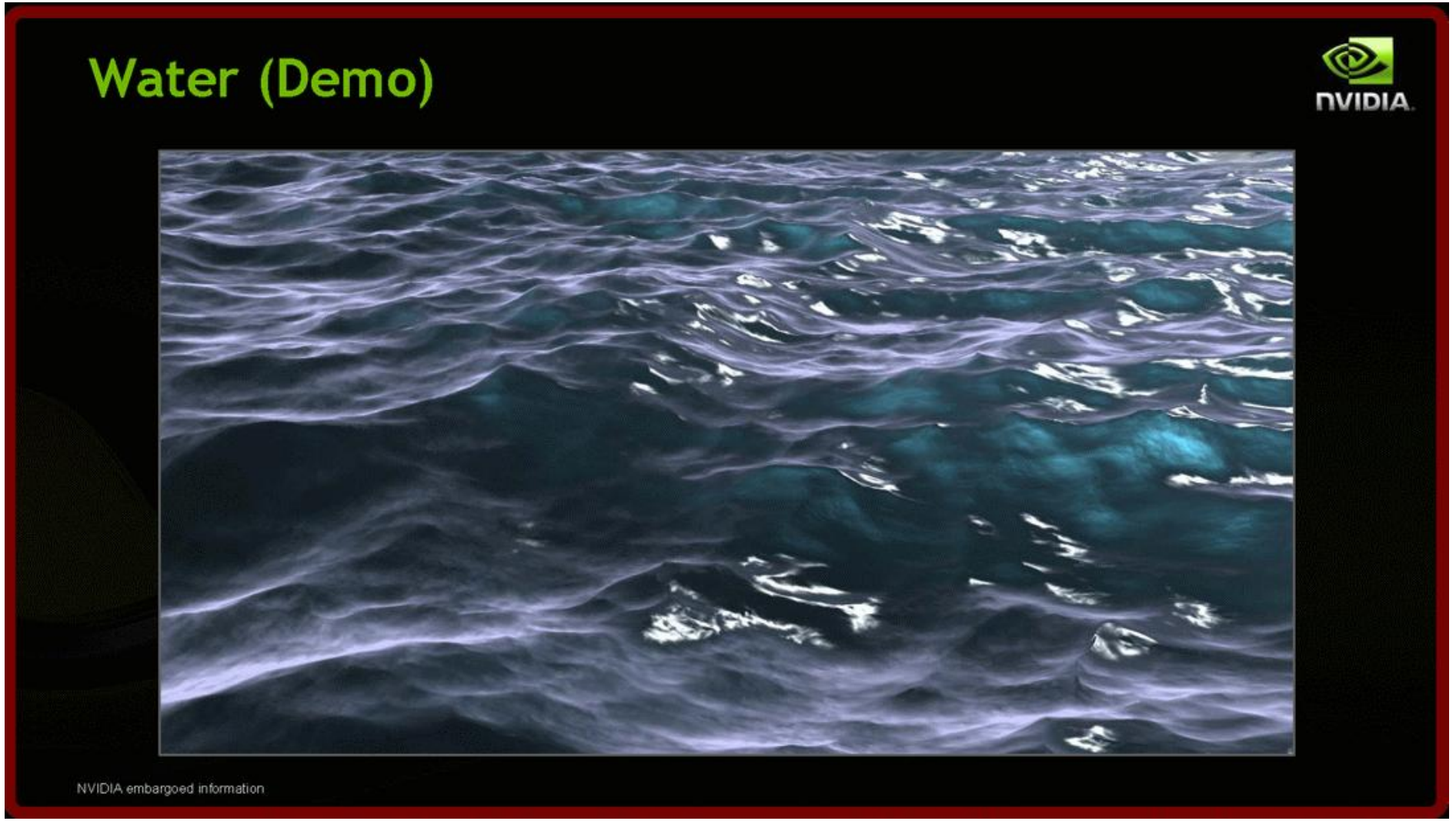
- Tessellation = no. of triangles



Display Map and Shading



Real-Time Water Flow



And…Hair

Hair (Demo)

NVIDIA

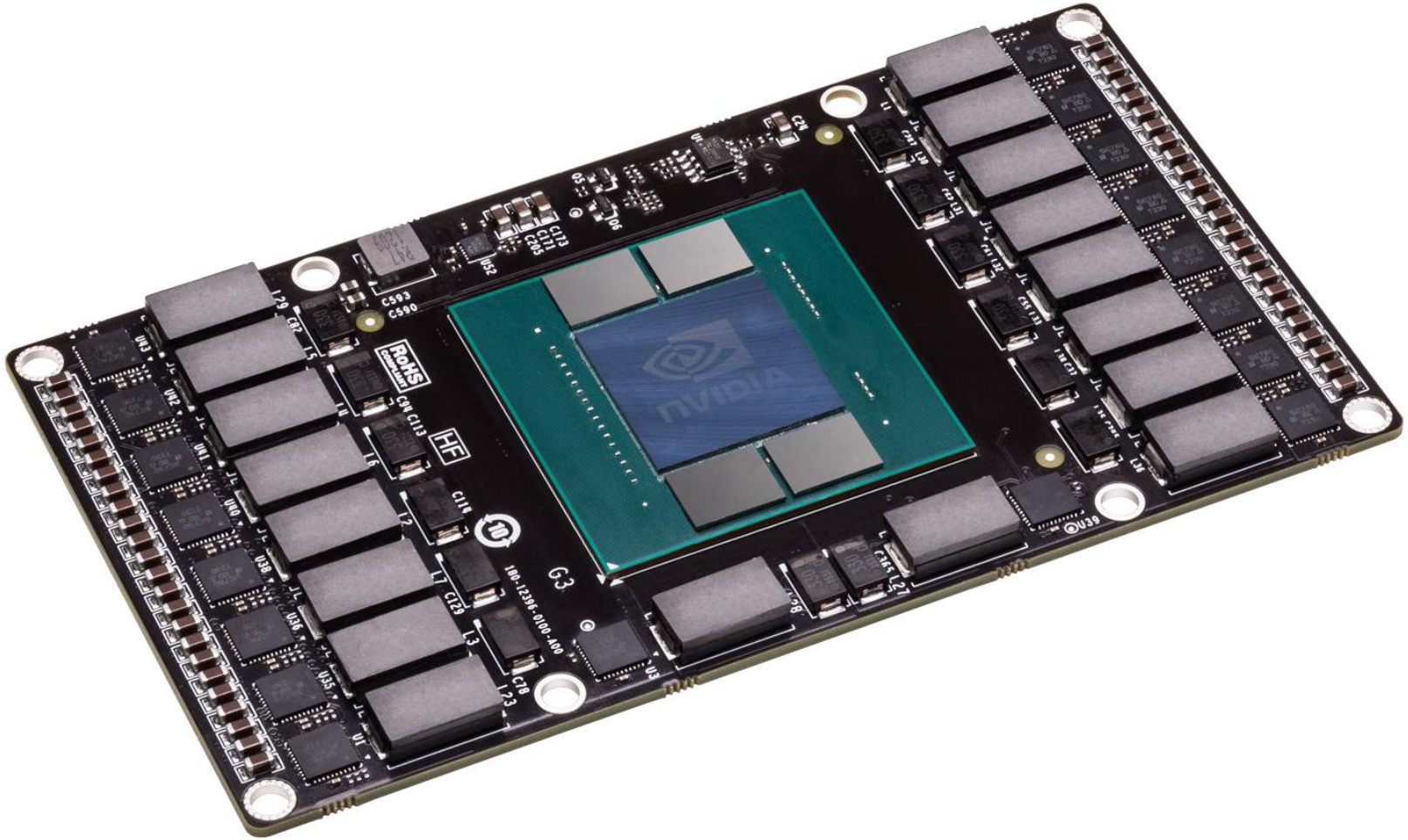
- Combine tessellation, geometry shading and compute to generate hair



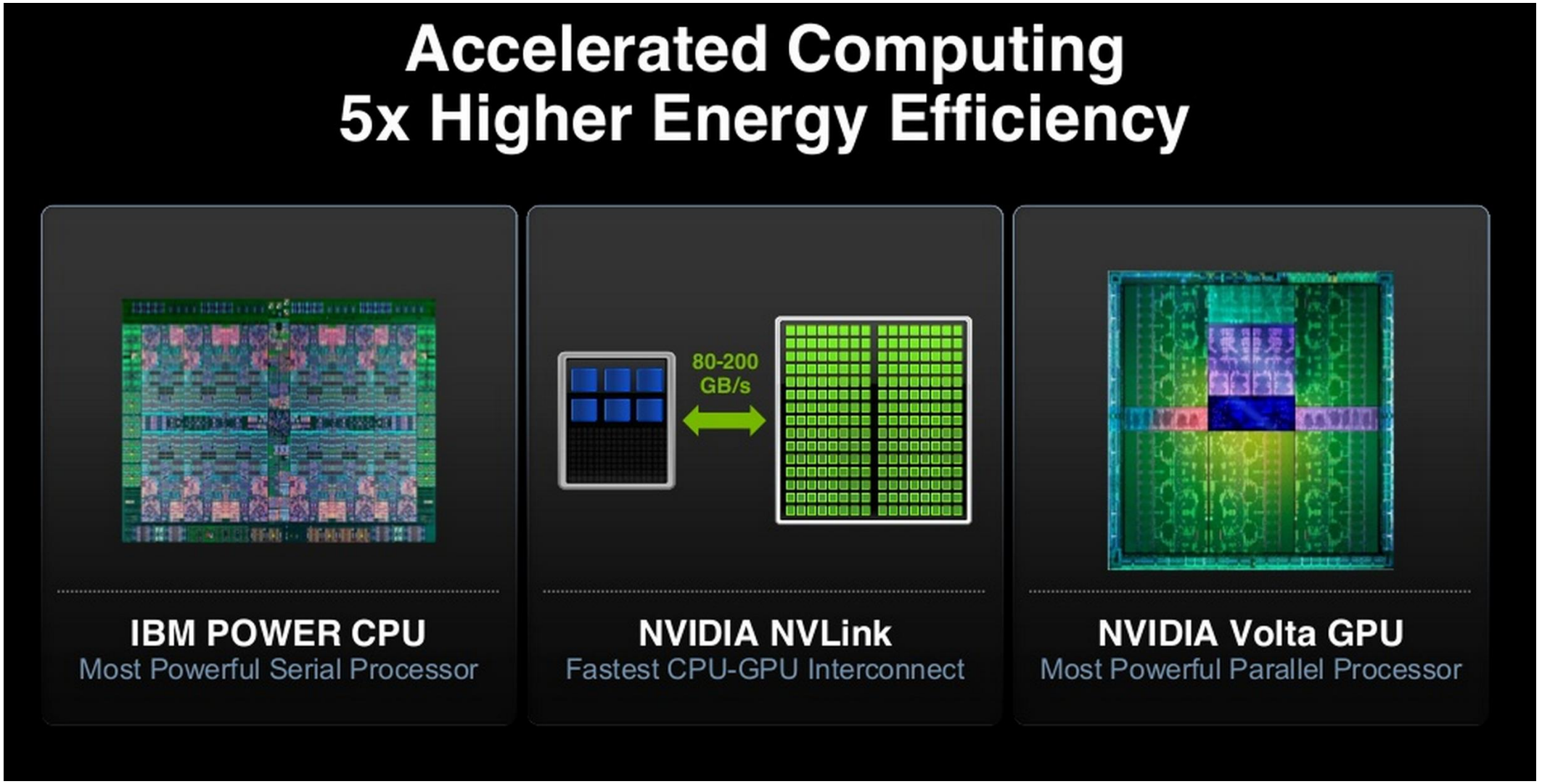
NVIDIA embargoed information

(demos)

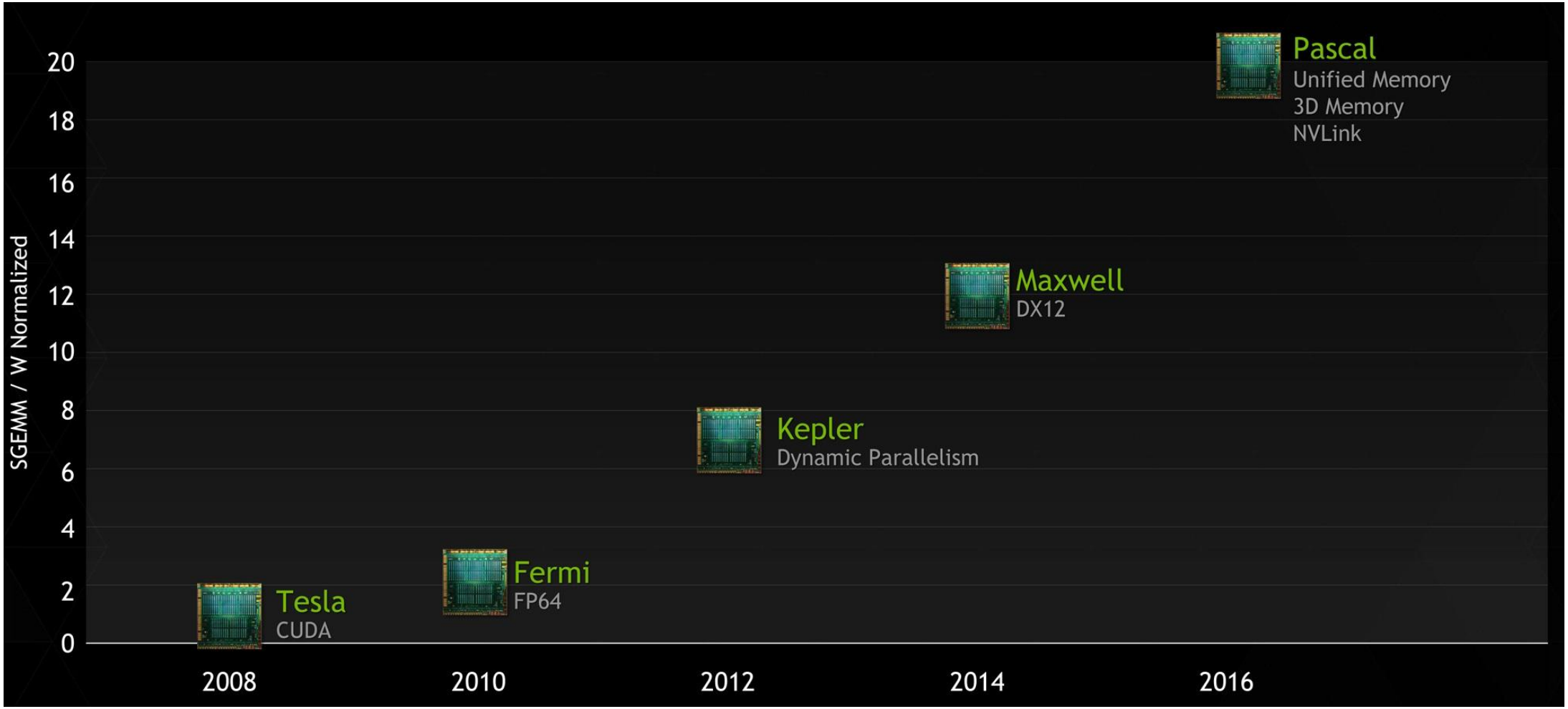
Next: Pascal = 10 x MAXWELL

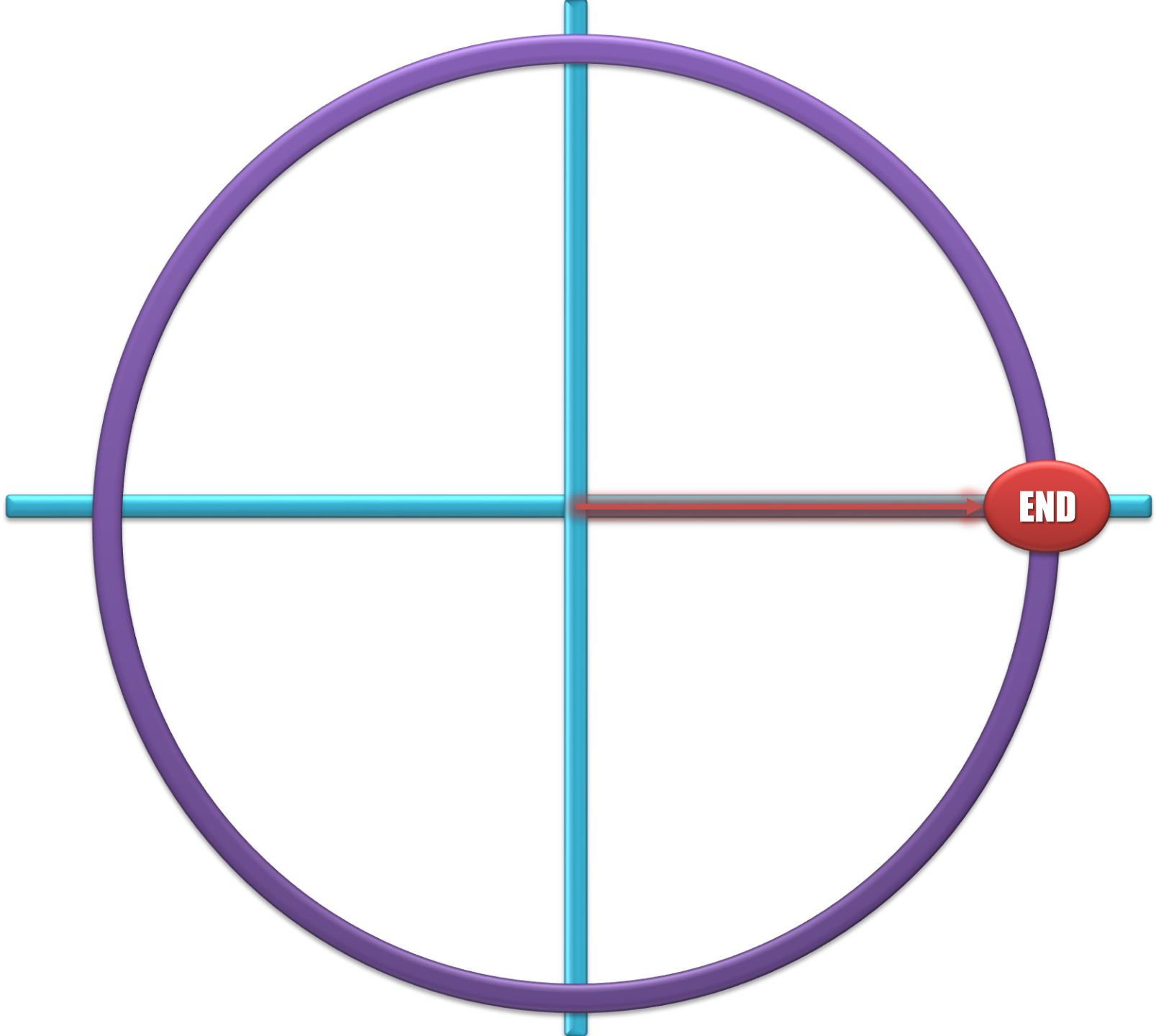


Next: Pascal = 10 x MAXWELL



SGEMM = Single Precision General Matrix Multiply




END

GPUs Beyond Graphics

- • CUDA / compute shaders / OpenCL: simulation, scientific computing, AI training & inference
- • The same throughput design that shades pixels trains neural networks
- • Tensor cores now serve BOTH pipelines: graphics (DLSS) and AI (LLMs)

What GPUs Mean for How You'll Code (week 6-11 preview)

- Shaders are small, stateless, branch-light programs — because of the SM design
- Uniforms = data all fragments share; varyings = interpolated per-fragment
- Draw calls are expensive, per-pixel work is (relatively) cheap

Every WebGL2 lab rule-of-thumb traces back to today's architecture

Week 2: The Three Takeaways

1. Framebuffer = the pixel array everything targets (you own one in TinyPipe from Lab 0)
2. Displays are raster scan-out devices — panels evolved (OLED/HDR/VRR), the abstraction didn't
3. GPU = massively parallel shader machine: SM/warp design IS the SIMD walkthrough, at 21,760-core scale

Revision hint: framebuffer math, display principles, and the SIMD argument are classic short-answer material

Next:

- Points vs vectors, homogeneous coordinates, the matrices behind everything
- Lab: Javascript, Setting up the DOM, Basic WebGL API, Graphics data flow,